

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: df = pd.read_csv("Medicaldataset.csv")
df.head()
```

```
Out[2]:
```

	Age	Gender	Heart rate	Systolic blood pressure	Diastolic blood pressure	Blood sugar	CK-MB	Troponin	Result
0	64	1	66	160	83	160.0	1.80	0.012	negative
1	21	1	94	98	46	296.0	6.75	1.060	positive
2	55	1	64	160	77	270.0	1.99	0.003	negative
3	64	1	70	120	55	270.0	13.87	0.122	positive
4	55	1	64	112	65	300.0	1.08	0.003	negative

```
In [3]: # Checking for values in each column
for col in df.columns:
    print(f"Unique value counts for column '{col}':")
    print(df[col].value_counts(dropna=False))
    print("\n")
```

Unique value counts for column 'Age':

Age

60 106

70 73

50 68

63 64

65 62

...

91 1

100 1

88 1

14 1

84 1

Name: count, Length: 75, dtype: int64

Unique value counts for column 'Gender':

Gender

1 870

0 449

Name: count, dtype: int64

Unique value counts for column 'Heart rate':

Heart rate

60 95

61 57

70 48

64 47

80 46

..

107 1

49 1

46 1

36 1

45 1

Name: count, Length: 79, dtype: int64

Unique value counts for column 'Systolic blood pressure':

Systolic blood pressure

150	50
130	41
125	41
120	38
140	32
..	
42	1
159	1
161	1
183	1
204	1

Name: count, Length: 116, dtype: int64

Unique value counts for column 'Diastolic blood pressure':

Diastolic blood pressure

75	69
68	67
80	57
70	44
65	43
..	
38	1
109	1
102	1
43	1
154	1

Name: count, Length: 73, dtype: int64

Unique value counts for column 'Blood sugar':

Blood sugar

100.0	41
87.0	32
98.0	31
96.0	30
94.0	30
..	
259.0	1
250.0	1
329.0	1

```
285.0      1
331.0      1
Name: count, Length: 244, dtype: int64
```

Unique value counts for column 'CK-MB':

```
CK-MB
300.000    19
1.600      11
1.330       9
1.830       9
2.930       9
..
0.973       1
33.700       1
0.483       1
6.410       1
25.040       1
```

Name: count, Length: 700, dtype: int64

Unique value counts for column 'Troponin':

```
Troponin
0.003    190
0.006     67
0.005     65
0.004     64
0.007     54
```

```
...
1.940       1
1.810       1
0.128       1
0.700       1
0.080       1
```

Name: count, Length: 352, dtype: int64

Unique value counts for column 'Result':

```
Result
positive    810
negative    509
```

Name: count, dtype: int64

Preprocessing and EDA

```
In [4]: df.isnull().sum() # Check for missing values
```

```
Out[4]: Age                0
Gender                0
Heart rate            0
Systolic blood pressure 0
Diastolic blood pressure 0
Blood sugar           0
CK-MB                 0
Troponin              0
Result                0
dtype: int64
```

```
In [5]: df.duplicated().sum() # Check for duplicates
```

```
Out[5]: np.int64(0)
```

```
In [6]: df.shape
```

```
Out[6]: (1319, 9)
```

```
In [7]: df['Result'] = df['Result'].map({'positive': 1, 'negative': 0})
```

```
In [8]: # Compute summary statistics for each risk group
summary_stats_risk_0 = df[df["Result"] == 0].describe()
summary_stats_risk_1 = df[df["Result"] == 1].describe()

# Display the statistics
print("Summary Statistics - Negative (0)")
display(summary_stats_risk_0)
```

```
print("\nSummary Statistics - Positive (1)")
display(summary_stats_risk_1)
```

Summary Statistics - Negative (0)

	Age	Gender	Heart rate	Systolic blood pressure	Diastolic blood pressure	Blood sugar	CK-MB	Troponin	Result
count	509.000000	509.000000	509.000000	509.000000	509.000000	509.000000	509.000000	509.000000	509.0
mean	52.094303	0.603143	77.886051	127.856582	72.440079	149.757760	2.555344	0.026988	0.0
std	13.730783	0.489727	48.211096	27.037031	14.325479	78.407363	1.367549	0.443320	0.0
min	14.000000	0.000000	20.000000	42.000000	40.000000	60.000000	0.321000	0.001000	0.0
25%	42.000000	0.000000	64.000000	110.000000	61.000000	98.000000	1.500000	0.003000	0.0
50%	52.000000	1.000000	75.000000	125.000000	72.000000	117.000000	2.310000	0.006000	0.0
75%	63.000000	1.000000	84.000000	147.000000	82.000000	184.000000	3.350000	0.009000	0.0
max	91.000000	1.000000	1111.000000	223.000000	128.000000	541.000000	7.020000	10.000000	0.0

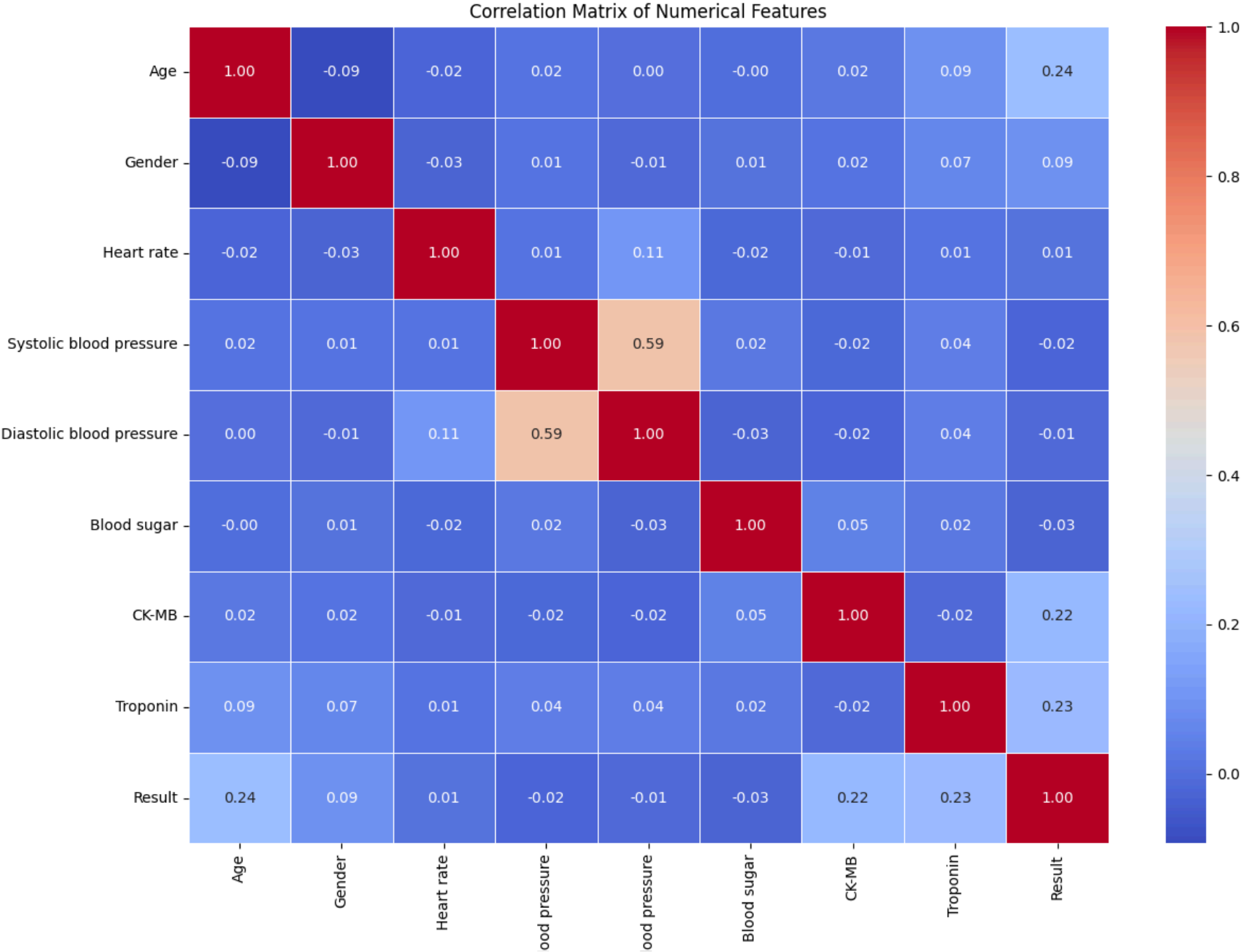
Summary Statistics - Positive (1)

	Age	Gender	Heart rate	Systolic blood pressure	Diastolic blood pressure	Blood sugar	CK-MB	Troponin	Result
count	810.000000	810.000000	810.000000	810.000000	810.000000	810.000000	810.000000	810.000000	810.0
mean	58.766667	0.695062	78.619753	126.739506	72.161728	144.671605	23.266838	0.570798	1.0
std	12.955419	0.460666	53.694817	25.538938	13.855417	72.628716	57.702774	1.390704	0.0
min	19.000000	0.000000	20.000000	65.000000	38.000000	35.000000	0.353000	0.003000	1.0
25%	50.000000	0.000000	64.000000	110.000000	62.000000	98.000000	1.870000	0.016000	1.0
50%	60.000000	1.000000	74.000000	122.000000	71.000000	116.000000	3.775000	0.044000	1.0
75%	68.000000	1.000000	85.750000	140.750000	80.000000	166.000000	12.250000	0.456250	1.0
max	103.000000	1.000000	1111.000000	223.000000	154.000000	541.000000	300.000000	10.300000	1.0

Class ratio: Negative (0) = 38.6% and Positive (1) = 61.4% (SMOTE will be considered to a more generalizability)

```
In [9]: # Compute the correlation matrix for numerical features
correlation_matrix = df.corr()

# Plot the heatmap
plt.figure(figsize=(14, 10))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", fmt=".2f", linewidths=0.5)
plt.title("Correlation Matrix of Numerical Features")
plt.show()
```



Systolic bp

Diastolic bp

Systolic BP and Diastolic BP have a high correlation value, hence only one **one to be implemented while modelling** or **create a new feature using both of them** : Pulse pressure (Systolic BP - Diastolic BP) which may help the models identify other patterns

Pulse pressure is the difference between the systolic pressure (the pressure in your arteries when your heart beats) and the diastolic pressure. Higher pulse pressure indicates stiff arteries and an increased risk of cardiovascular diseases.

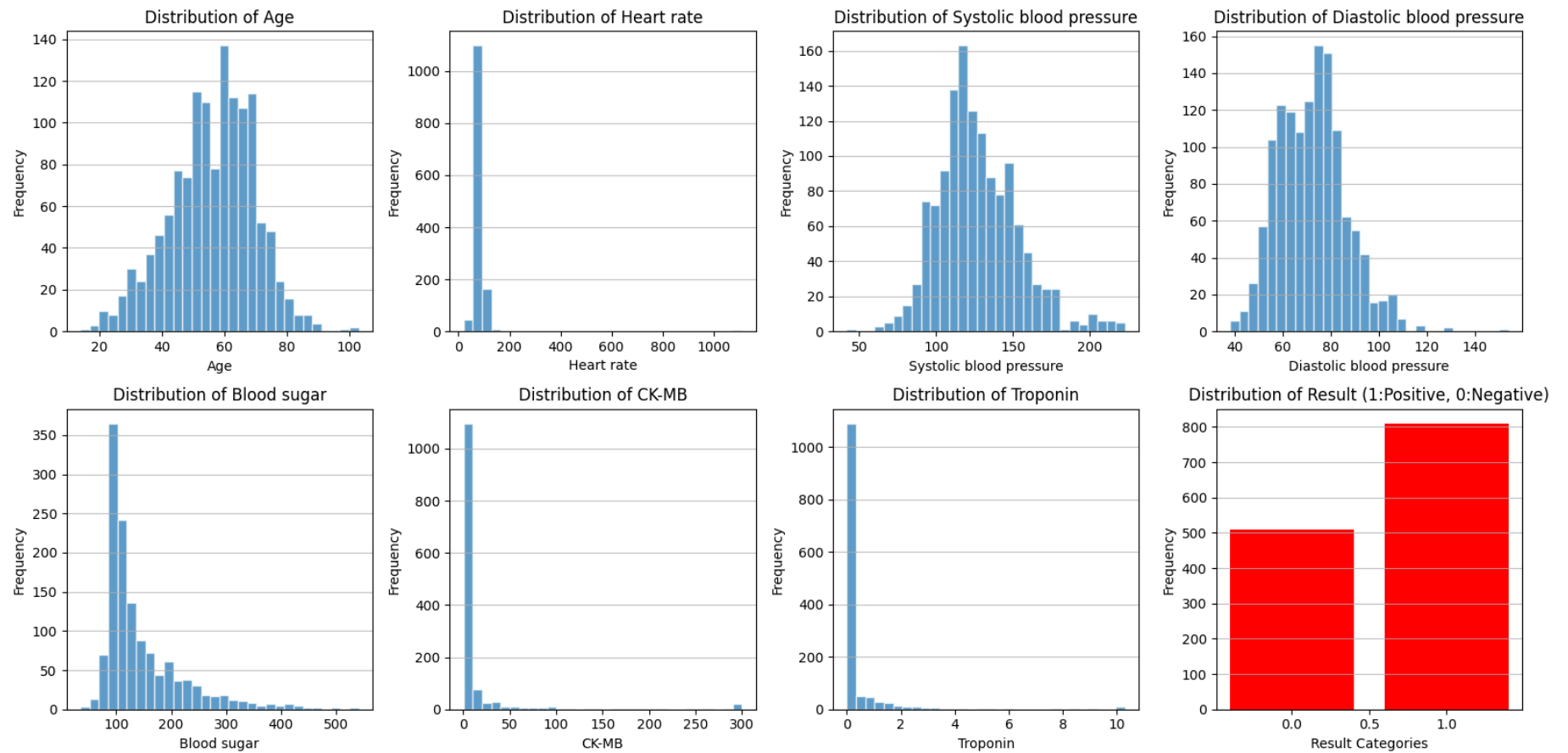
```
In [10]: df['Pulse Pressure'] = df['Systolic blood pressure'] - df['Diastolic blood pressure']
```

```
In [11]: vars = ["Age", "Heart rate", "Systolic blood pressure", "Diastolic blood pressure", "Blood sugar", "CK-MB", "Troponin"]
```

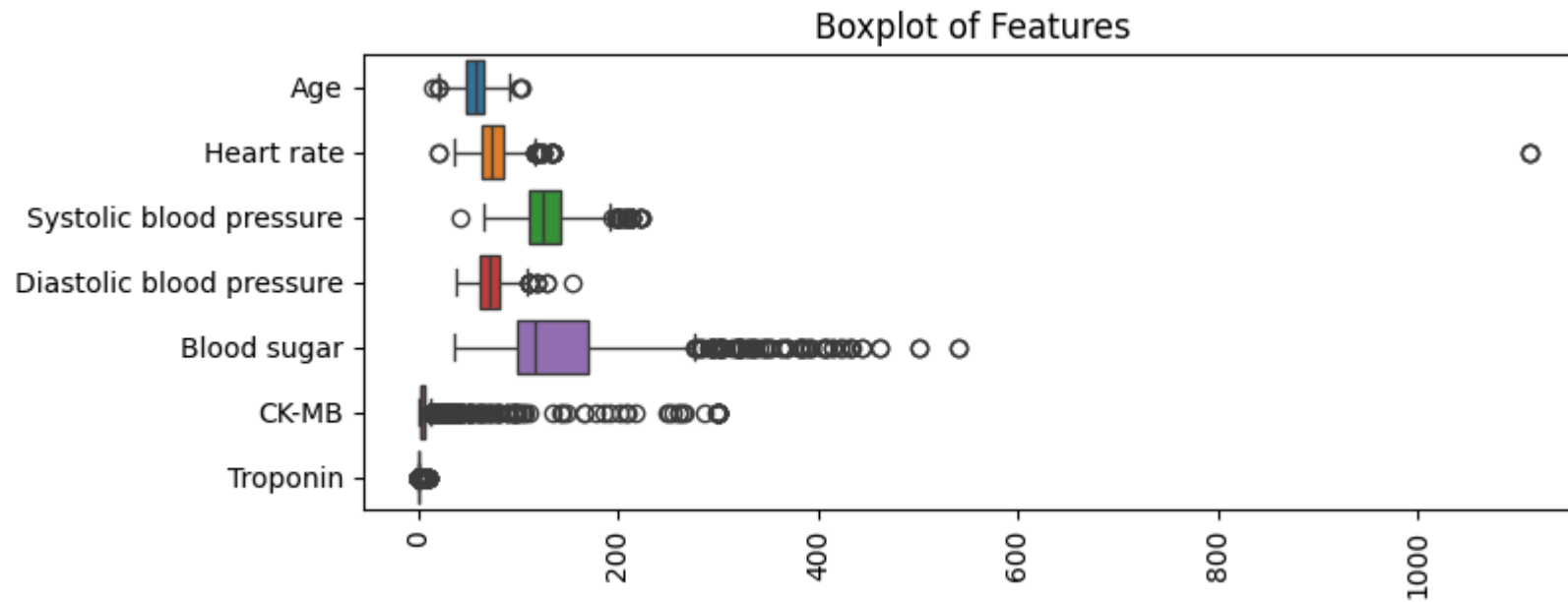
```
fig, axes = plt.subplots(nrows=2, ncols=4, figsize=(16, 8))
axes = axes.flatten()
# Create histograms for each variable
for i, var in enumerate(vars):
    axes[i].hist(df[var], bins=30, edgecolor='white', alpha=0.7)
    axes[i].set_title(f"Distribution of {var}")
    axes[i].set_xlabel(var)
    axes[i].set_ylabel("Frequency")
    axes[i].grid(axis='y', alpha=0.75)

result_counts = df["Result"].value_counts()
axes[-1].bar(result_counts.index, result_counts.values, color='red') # Bar chart
axes[-1].set_title("Distribution of Result (1:Positive, 0:Negative)")
axes[-1].set_xlabel("Result Categories")
axes[-1].set_ylabel("Frequency")
axes[-1].grid(axis='y', alpha=0.75)

plt.tight_layout()
plt.show()
```



```
In [12]: box_vars = ["Age", "Heart rate", "Systolic blood pressure", "Diastolic blood pressure", "Blood sugar", "CK-MB", "Troponin"]
plt.figure(figsize=(8, 3))
sns.boxplot(data=df[box_vars], orient="h")
plt.xticks(rotation=90)
plt.title("Boxplot of Features")
plt.show()
```

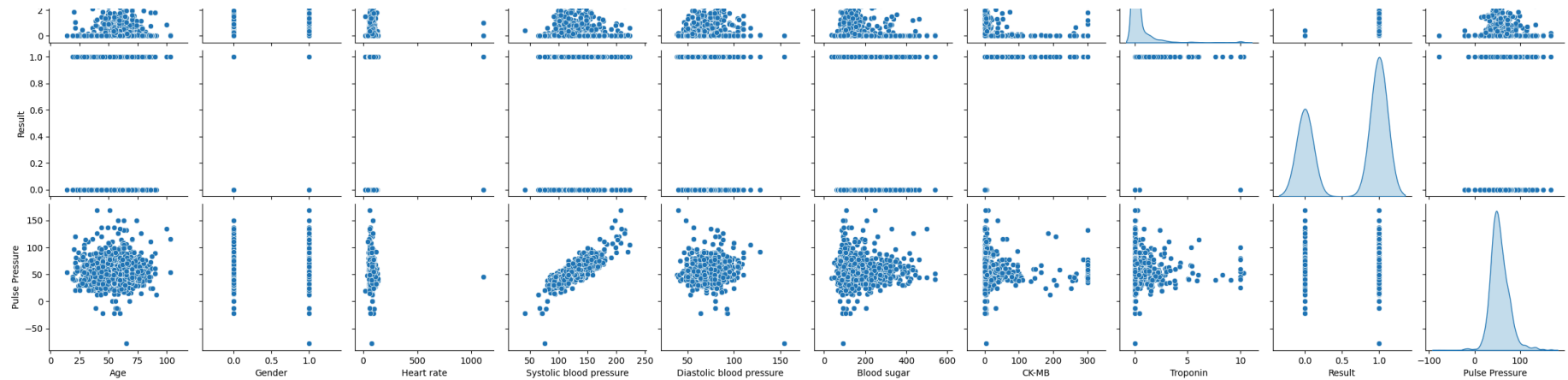


Will use RobustScaler since Blood Sugar, CK-MB and Troponin contain many outliers StandardScaler would have sufficed for Systolic BP and Diastolic BP RobustScaler but would ensure all the features are on similar scale

EDA

```
In [13]: sns.pairplot(df, diag_kind="kde")  
plt.show()
```

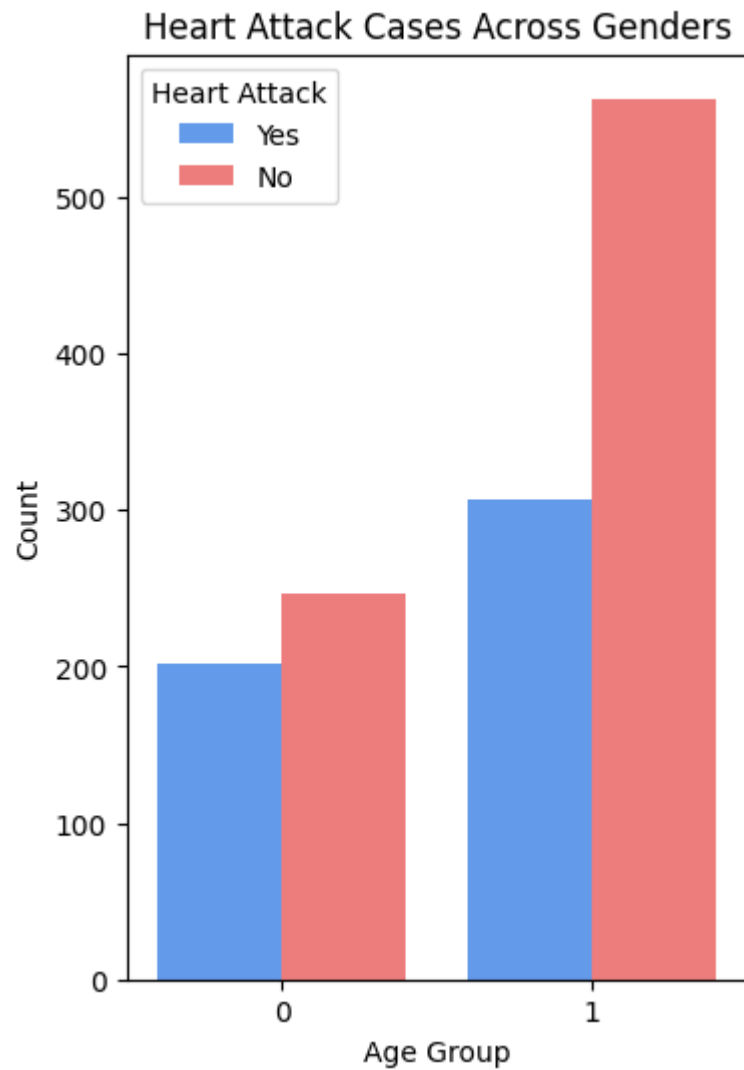




```
In [14]: # Define better colors with more contrast
colors = {"Yes": "#FF6B6B", "No": "#4D96FF"} # Red for disease, Blue for no disease
df['Heart_Attack'] = df['Result'].map({1: "Yes", 0: "No"})
```

```
In [15]: plt.figure(figsize=(4, 6))
sns.countplot(x=df['Gender'], hue=df['Heart_Attack'], data=df, palette=colors) # Remove quotes around colors

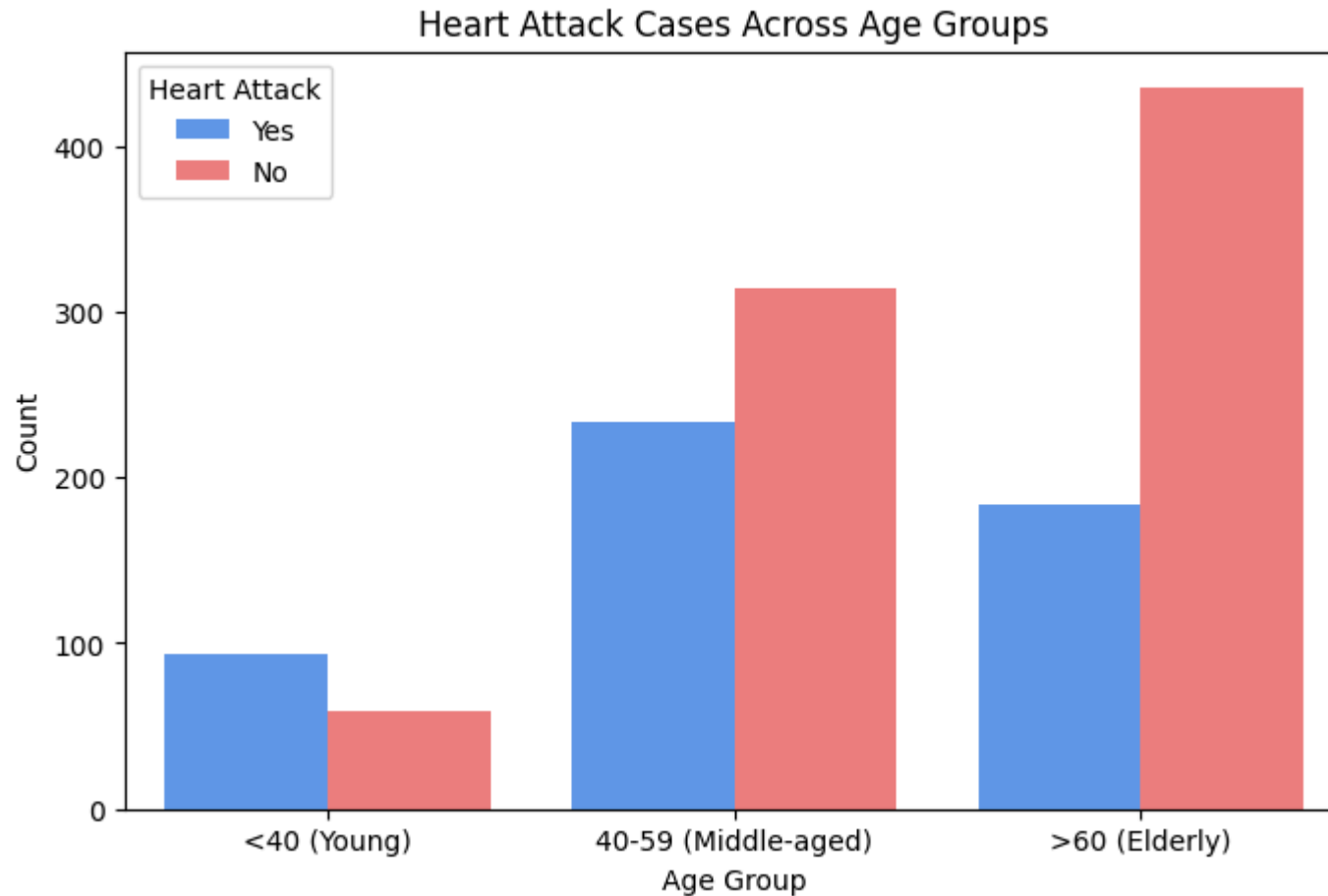
plt.title("Heart Attack Cases Across Genders")
plt.xlabel("Age Group") # Note: This should probably be "Gender" not "Age Group"
plt.ylabel("Count")
plt.legend(title="Heart Attack", labels=["Yes", "No"])
plt.show()
```



```
In [16]: # Define age groups
bins = [0, 39, 59, 100] # Ranges
labels = ["<40 (Young)", "40-59 (Middle-aged)", ">60 (Elderly)"]
df["Age Group"] = pd.cut(df["Age"], bins=bins, labels=labels)

plt.figure(figsize=(8, 5))
sns.countplot(x="Age Group", hue="Heart_Attack", data=df, palette=colors)
```

```
plt.title("Heart Attack Cases Across Age Groups")
plt.xlabel("Age Group")
plt.ylabel("Count")
plt.legend(title="Heart Attack", labels=["Yes", "No"])
plt.show()
```

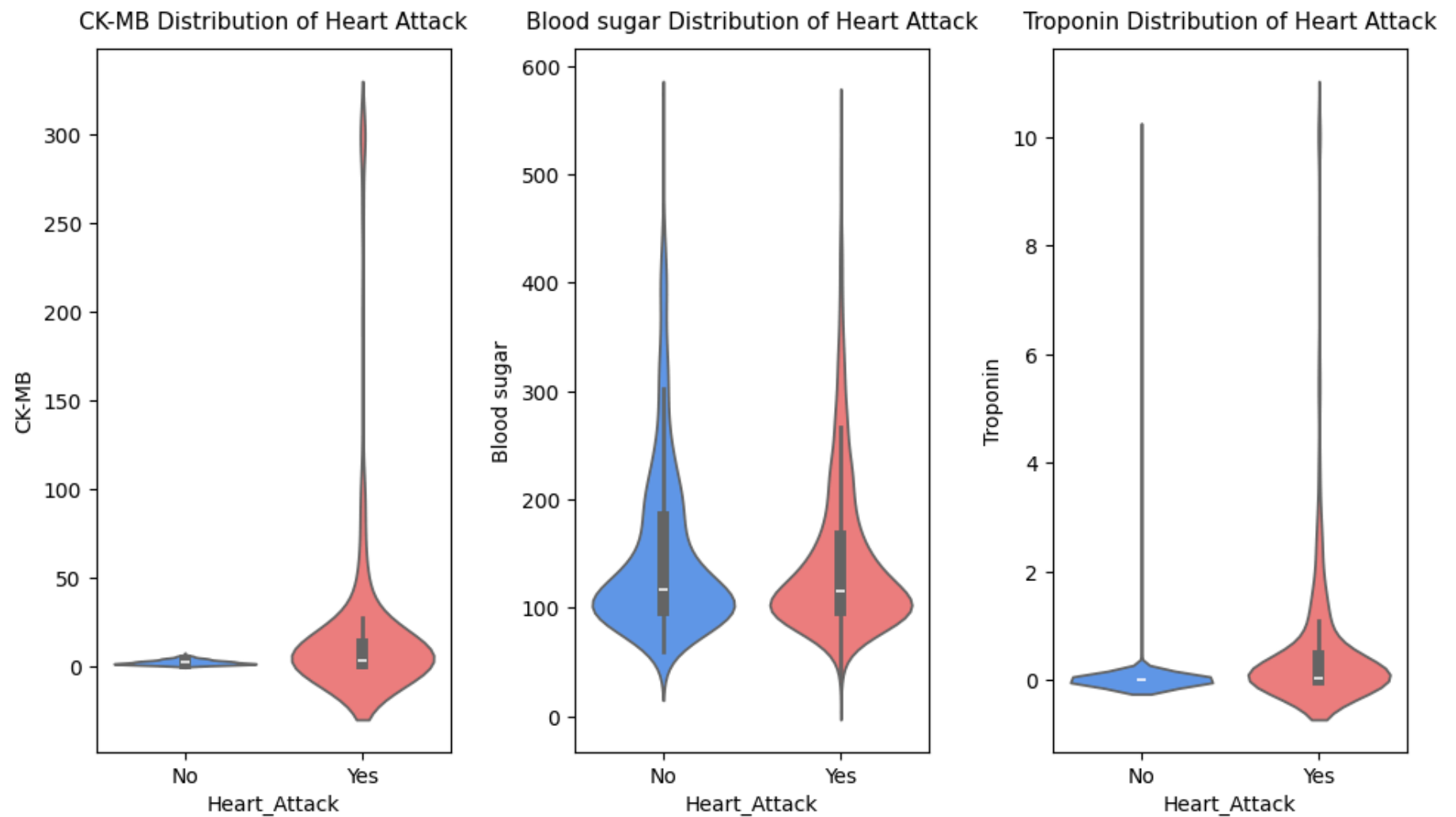


```
In [17]: bio_features = ["CK-MB", "Blood sugar", "Troponin"]
```

```
# Create a figure with subplots
plt.figure(figsize=(10, 6))
for i, feature in enumerate(bio_features, 1):
```

```
plt.subplot(1, 3, i)
sns.violinplot(x="Heart_Attack", y=feature, data=df, hue="Heart_Attack", palette=colors, inner="box", legend=False)
plt.title(f"{feature} Distribution of Heart Attack", fontsize=11, pad=10)
plt.xlabel("Heart_Attack")

plt.tight_layout(pad=2.0) # Increase padding
plt.show()
```



AI Prompt to generate selected EDA graphs in one layout: "Combine all these graphs together into one layout on python" and provided copies of the graphs

```
In [18]: import matplotlib.gridspec as gridspec

fig = plt.figure(figsize=(16, 12))
gs = gridspec.GridSpec(2, 3)

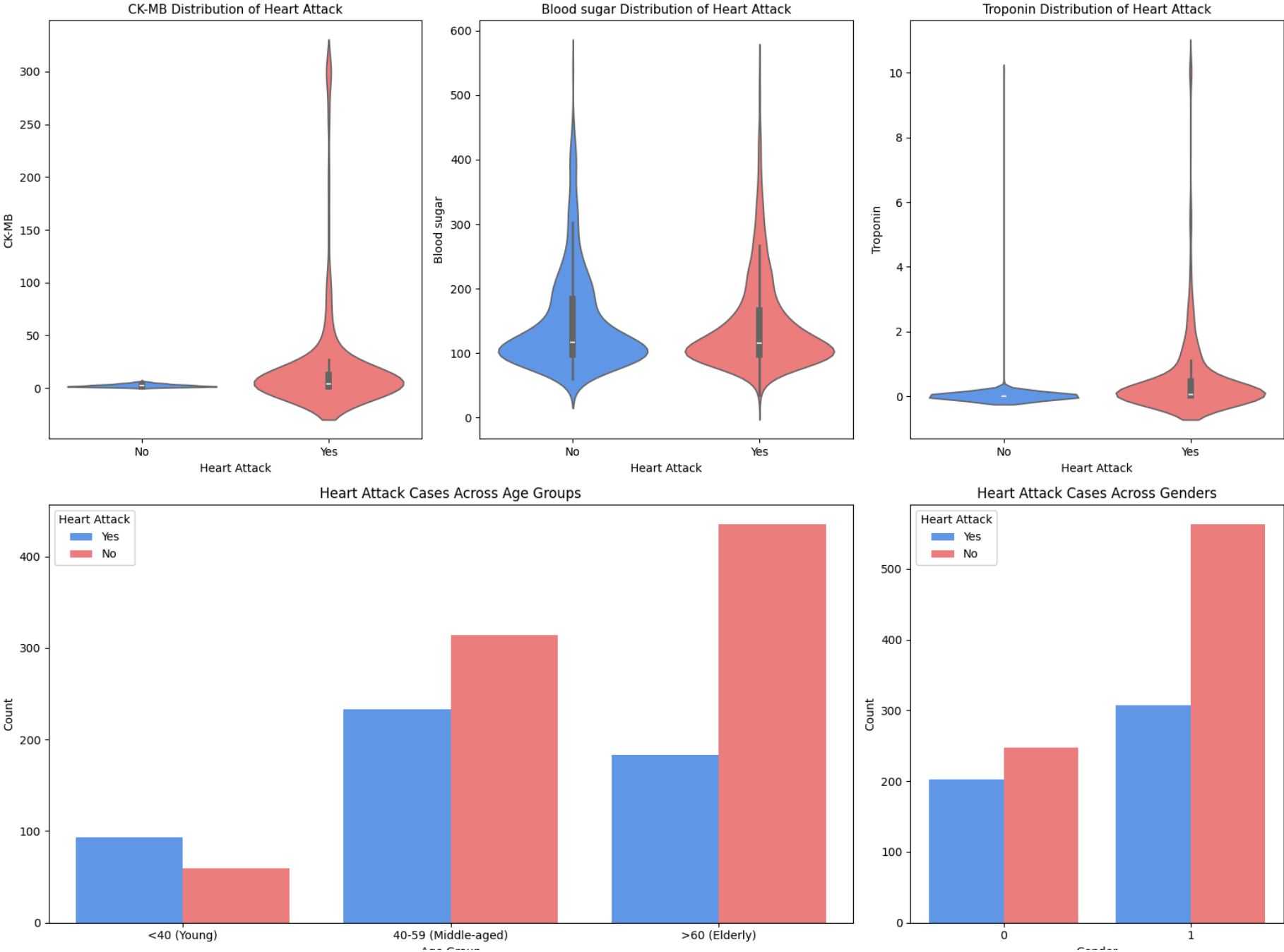
# Top row: The three violin plots
for i, feature in enumerate(bio_features):
    ax = fig.add_subplot(gs[0, i])
    sns.violinplot(x="Heart_Attack", y=feature, data=df,
                  hue="Heart_Attack", palette=colors, inner="box", legend=False)
    ax.set_title(f"{feature} Distribution of Heart Attack", fontsize=11)
    ax.set_xlabel("Heart Attack")

# Bottom Left: Heart Attack by Age Group
ax1 = fig.add_subplot(gs[1, 0:2]) # Span two columns
sns.countplot(x="Age Group", hue="Heart_Attack", data=df, palette=colors)
ax1.set_title("Heart Attack Cases Across Age Groups")
ax1.set_xlabel("Age Group")
ax1.set_ylabel("Count")
ax1.legend(title="Heart Attack", labels=["Yes", "No"])

# Bottom right: Heart Attack by Gender
ax2 = fig.add_subplot(gs[1, 2])
sns.countplot(x="Gender", hue="Heart_Attack", data=df, palette=colors)
ax2.set_title("Heart Attack Cases Across Genders")
ax2.set_xlabel("Gender")
ax2.set_ylabel("Count")
ax2.legend(title="Heart Attack", labels=["Yes", "No"])

plt.tight_layout()
fig.suptitle("Heart Attack Data Analysis", fontsize=16, y=1.02)
plt.show()
```

Heart Attack Data Analysis



MODELLING

```
In [19]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import RobustScaler
from sklearn.svm import SVC
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.metrics import accuracy_score, classification_report, roc_auc_score, confusion_matrix, roc_curve, auc, ConfusionM
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.inspection import permutation_importance
from sklearn import metrics
```

```
In [20]: import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, TensorDataset
from sklearn.model_selection import KFold
import time
```

```
In [21]: # features
X1 = ['Age', 'Gender', 'Heart rate', 'Systolic blood pressure', 'Blood sugar', 'CK-MB', 'Troponin']
X2 = ['Age', 'Gender', 'Heart rate', 'Pulse Pressure', 'Blood sugar', 'CK-MB', 'Troponin']

# target
y = 'Result'
```

```
In [22]: X1_train, X1_test, y1_train, y1_test = train_test_split(df[X1], df[y], test_size=0.2, random_state=42)
X2_train, X2_test, y2_train, y2_test = train_test_split(df[X2], df[y], test_size=0.2, random_state=42)
```

```
In [23]: # Define the scaler
scaler1 = RobustScaler()
scaler2 = RobustScaler()

# Fit and transform the training data
X1_train_scaled = scaler1.fit_transform(X1_train)
X2_train_scaled = scaler2.fit_transform(X2_train)
```

```
X1_test_scaled = scaler1.transform(X1_test)
X2_test_scaled = scaler2.transform(X2_test)
```

Baseline Models

SVM

```
In [24]: svm_basemodel = SVC() # Default SVM with RBF kernel
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42) # Ensures balanced class distribution
# Perform 10-Fold Cross-Validation
cv_scores1 = cross_val_score(svm_basemodel, X1_train, y1_train, cv=cv, scoring='accuracy')
print(f"SVM (Systolic BP) 10-Fold CV Accuracy: {np.mean(cv_scores1):.4f}")

cv_scores2 = cross_val_score(svm_basemodel, X2_train, y2_train, cv=cv, scoring='accuracy')
print(f"SVM (Pulse Pressure) 10-Fold CV Accuracy: {np.mean(cv_scores2):.4f}")
```

SVM (Systolic BP) 10-Fold CV Accuracy: 0.6607

SVM (Pulse Pressure) 10-Fold CV Accuracy: 0.6664

MLP

```
In [25]: mlp_basemodel = MLPClassifier(random_state=42)
cv_scores1 = cross_val_score(mlp_basemodel, X1_train, y1_train, cv=cv, scoring='accuracy')
print(f"MLP (Systolic BP) 10-Fold CV Accuracy: {np.mean(cv_scores1):.4f}")

cv_scores2 = cross_val_score(mlp_basemodel, X2_train, y2_train, cv=cv, scoring='accuracy')
print(f"MLP (Pulse Pressure) 10-Fold CV Accuracy: {np.mean(cv_scores2):.4f}")
```

c:\Users\sajja\anaconda3\envs\neuralcomp\env\Lib\site-packages\sklearn\network_multilayer_perceptron.py:691: Convergence Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.

```
warnings.warn(
MLP (Systolic BP) 10-Fold CV Accuracy: 0.7557
```

```
c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
```

MLP (Pulse Pressure) 10-Fold CV Accuracy: 0.7586

Despite not reaching convergence, the MLP models outperform SVM. Pulse pressure as the feature is yielding better performance compared to Systolic Blood Pressure. To mitigate the issue of iteration, we may train the model with a maximum iterations of 1000 or implement hyperparameter tuning through Grid search

Feature Selection

From the baseline, we will attempt to reduce the number of features to improve generalizability of the models; we will focus only on the model with 'Pulse pressure'. We will use 'permutation_importance' to identify the impact of the features since SVM/MLP do not have a direct feature importance option

```
In [26]: svm_model_fs = SVC(probability=True, random_state=42)
svm_model_fs.fit(X2_train, y2_train)

# Permutation importance for SVM
result = permutation_importance(svm_model_fs, X2_train, y2_train, n_repeats=10, random_state=42)
svm_importances = result.importances_mean

# Display results
print("Feature Importances (SVM):")
for i, feature in enumerate(X2):
    print(f"{feature}: {svm_importances[i]:.4f}")

mlp_model_fs = MLPClassifier(random_state=42)
mlp_model_fs.fit(X2_train, y2_train)

# Permutation importance for MLP
result = permutation_importance(mlp_model_fs, X2_train, y2_train, n_repeats=10, random_state=42)
```

```
mlp_importances = result.importances_mean

# Display results
print("Feature Importances (MLP):")
for i, feature in enumerate(X2):
    print(f"{feature}: {mlp_importances[i]:.4f}")
```

Feature Importances (SVM):

Age: 0.0775
 Gender: 0.0002
 Heart rate: -0.0018
 Pulse Pressure: -0.0016
 Blood sugar: 0.0043
 CK-MB: 0.0407
 Troponin: 0.0020
 Feature Importances (MLP):
 Age: 0.0461
 Gender: 0.0061
 Heart rate: 0.0037
 Pulse Pressure: 0.0188
 Blood sugar: 0.0118
 CK-MB: 0.1364
 Troponin: 0.0973

Age is an important factor in both models. CK-MB and Troponin are impactful especially in MLP. Different patterns are observed from the two models.

```
In [27]: # Feature reduction
reduced_features = ['Age', 'Pulse Pressure', 'Blood sugar', 'CK-MB', 'Troponin']
important_features = ['Age', 'CK-MB', 'Troponin']

X_reduced_train, X_reduced_test, y_reduced_train, y_reduced_test = train_test_split(df[reduced_features], df['Result'], test_size=0.2, random_state=42)
X_impt_train, X_impt_test, y_impt_train, y_impt_test = train_test_split(df[important_features], df['Result'], test_size=0.2, random_state=42)

scaler3 = RobustScaler()
scaler4 = RobustScaler()

X_reduced_train_scaled = scaler3.fit_transform(X_reduced_train)
X_impt_train_scaled = scaler4.fit_transform(X_impt_train)
```

```
X_reduced_test_scaled = scaler3.transform(X_reduced_test)
X_impt_test_scaled = scaler4.transform(X_impt_test)
```

SVM Model with reduced features

```
In [28]: svm_model = SVC() # Default SVM with RBF kernel
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42) # Ensures balanced class distribution
# Perform 10-Fold Cross-Validation
cv_scores3 = cross_val_score(svm_model, X_reduced_train_scaled, y_reduced_train, cv=cv, scoring='accuracy')
print(f"SVM (Reduced feactures) 10-Fold CV Accuracy: {np.mean(cv_scores3):.4f}")

cv_scores4 = cross_val_score(svm_model, X_impt_train, y_impt_train, cv=cv, scoring='accuracy')
print(f"SVM (Most important features) 10-Fold CV Accuracy: {np.mean(cv_scores2):.4f}")
```

SVM (Reduced feactures) 10-Fold CV Accuracy: 0.7888

SVM (Most important features) 10-Fold CV Accuracy: 0.7586

MLP Model with reduced features

```
In [29]: mlp_model = MLPClassifier(random_state=42)
cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=42) # Ensures balanced class distribution
# Perform 10-Fold Cross-Validation
cv_scores3 = cross_val_score(mlp_model, X_reduced_train_scaled, y_reduced_train, cv=cv, scoring='accuracy')
print(f"MLP (Reduced feactures) 10-Fold CV Accuracy: {np.mean(cv_scores3):.4f}")

cv_scores4 = cross_val_score(mlp_model, X_impt_train, y_impt_train, cv=cv, scoring='accuracy')
print(f"MLP (Most important features) 10-Fold CV Accuracy: {np.mean(cv_scores2):.4f}")
```

```
c:\Users\sajja\anaconda3\envs\nuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\nuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\nuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\nuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\nuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\nuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\nuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\nuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\nuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
MLP (Reduced feactures) 10-Fold CV Accuracy: 0.9470
```

```

c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
MLP (Most important features) 10-Fold CV Accuracy: 0.7586
c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:691: Convergence
Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(

```

The models with the smaller set of features (reduced_features) show great improvement in the performance the models. MLP failed to converge again which we expect to be solved during hyperparameter optimisation. An accuracy for MLP of 94.7% is observed.

Hyperparameter Tuning (Grid Search)

SVM Grid Search

```

In [30]: # Parameter grid for SVM optimisation
param_grid = {

```

```

    'C': [0.1, 1, 10, 100], # Regularization parameter
    'gamma': ['scale', 'auto', 0.1, 0.01], # Kernel coefficient
    'kernel': ['rbf', 'linear'] # Kernel
}

svm = SVC(random_state=42)

grid_search = GridSearchCV(
    estimator=svm,
    param_grid=param_grid,
    cv=cv, # Using existing StratifiedKFold
    scoring='accuracy',
    n_jobs=-1, # Using maximum number of CPU cores
    verbose=1
)

grid_search.fit(X_reduced_train_scaled, y_reduced_train)

# Print the best parameters and score
print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.4f}")

```

Fitting 10 folds for each of 32 candidates, totalling 320 fits

Best parameters: {'C': 100, 'gamma': 0.1, 'kernel': 'rbf'}

Best cross-validation score: 0.9593

```

In [31]: results_SVM = pd.DataFrame(grid_search.cv_results_)

params_results = results_SVM[['param_C', 'param_kernel', 'param_gamma', 'mean_test_score', 'rank_test_score']]
params_results = params_results.sort_values(by='rank_test_score')

# Print or display the dataframe
print(params_results)

```

	param_C	param_kernel	param_gamma	mean_test_score	rank_test_score
28	100.0	rbf	0.1	0.959281	1
26	100.0	rbf	auto	0.957385	2
18	10.0	rbf	auto	0.948895	3
20	10.0	rbf	0.1	0.946065	4
17	10.0	linear	scale	0.943226	5
19	10.0	linear	auto	0.943226	5
21	10.0	linear	0.1	0.943226	5
23	10.0	linear	0.01	0.943226	5
25	100.0	linear	scale	0.942282	9
27	100.0	linear	auto	0.942282	9
29	100.0	linear	0.1	0.942282	9
31	100.0	linear	0.01	0.942282	9
30	100.0	rbf	0.01	0.937574	13
13	1.0	linear	0.1	0.933792	14
11	1.0	linear	auto	0.933792	14
15	1.0	linear	0.01	0.933792	14
9	1.0	linear	scale	0.933792	14
24	100.0	rbf	scale	0.928095	18
22	10.0	rbf	0.01	0.911968	19
12	1.0	rbf	0.1	0.902471	20
10	1.0	rbf	auto	0.900575	21
1	0.1	linear	scale	0.892066	22
3	0.1	linear	auto	0.892066	22
5	0.1	linear	0.1	0.892066	22
7	0.1	linear	0.01	0.892066	22
16	10.0	rbf	scale	0.873109	26
2	0.1	rbf	auto	0.844654	27
14	1.0	rbf	0.01	0.837071	28
4	0.1	rbf	0.1	0.828518	29
8	1.0	rbf	scale	0.788760	30
0	0.1	rbf	scale	0.788589	31
6	0.1	rbf	0.01	0.739461	32

MLP Grid Search

SK-Learn MLP Classifier

```
In [32]: # Parmeter grid for MLP optimisation
param_grid = {
```

```

    'hidden_layer_sizes': [(20,), (50,), (20, 10), (50, 20), (50, 50), (100, 50)], # Number of neurons in each hidden layer
    'activation': ['relu', 'tanh'], # Activation functions
    'solver': ['adam', 'sgd'], # Backpropagation algorithm
    'alpha': [0.0001, 0.001, 0.01], # Regularization
    'learning_rate_init': [0.001, 0.01], # Learning rates
    'max_iter': [500] # Increased max iterations for convergence
}

mlp = MLPClassifier(random_state=42)

grid_search = GridSearchCV(
    estimator=mlp,
    param_grid=param_grid,
    cv=cv,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_reduced_train_scaled, y_reduced_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.4f}")

```

Fitting 10 folds for each of 144 candidates, totalling 1440 fits

Best parameters: {'activation': 'tanh', 'alpha': 0.0001, 'hidden_layer_sizes': (20, 10), 'learning_rate_init': 0.001, 'max_iter': 500, 'solver': 'adam'}

Best cross-validation score: 0.9744

c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (500) reached and the optimization hasn't converged yet.
warnings.warn(

MLP failed to converge at 500 maximum iterations. We will test the model on the Test Set and decide whether to increase maximum iterations or not

Fitting 10 folds for each of 96 candidates, totalling 960 fits Best parameters: {'activation': 'tanh', 'alpha': 0.001, 'hidden_layer_sizes': (100, 50), 'learning_rate_init': 0.001, 'max_iter': 1000, 'solver': 'adam'} Best cross-validation score: 0.9725

c:\Users\sajja\anaconda3\envs\neuralcompenv\Lib\site-packages\sklearn\neural_network_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet. warnings.warn(

Failed to converge with maximum iterations of 1000 Ran the grid search with 1250 and the MLP converges. We kept reducing the maximum iterations and found out that the model converges with 1005 iterations with an accuracy 0.9725

```
In [33]: # Parmeter grid for MLP optimisation (Increased max iterations since previous best model show no sign of overfitting when test
param_grid = {
    'hidden_layer_sizes': [(50,), (100,), (50, 50), (100, 50)], # Number of neurons in each hidden layer
    'activation': ['relu', 'tanh'], # Activation functions
    'solver': ['adam', 'sgd'], # Backpropagation algorithm
    'alpha': [0.0001, 0.001, 0.01], # Regularization
    'learning_rate_init': [0.001, 0.01], # Learning rates
    'max_iter': [500, 750, 1005], # Increased max iterations for convergence
    'early_stopping': [True] # Early stopping to prevent overfitting and stop model training when validation score is not impr
}

mlp = MLPClassifier(random_state=42)

grid_search = GridSearchCV(
    estimator=mlp,
    param_grid=param_grid,
    cv=cv,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_reduced_train_scaled, y_reduced_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.4f}")
```

Fitting 10 folds for each of 288 candidates, totalling 2880 fits

Best parameters: {'activation': 'relu', 'alpha': 0.0001, 'early_stopping': True, 'hidden_layer_sizes': (100, 50), 'learning_rate_init': 0.01, 'max_iter': 500, 'solver': 'adam'}

Best cross-validation score: 0.9603

```
In [34]: # Parmeter grid for MLP optimisation (Increased max iterations since previous best model show no sign of overfitting when test
param_grid = {
    'hidden_layer_sizes': [(20,), (50,), (20, 10), (50, 20)], # Number of neurons in each hidden layer
    'activation': ['relu', 'tanh'], # Activation functions
    'solver': ['adam', 'sgd'], # Backpropagation algorithm
```

```

    'alpha': [0.0001, 0.001, 0.01], # Regularization
    'learning_rate_init': [0.001, 0.01], # Learning rates
    'max_iter': [500, 750, 1005], # Increased max iterations for convergence
    'early_stopping': [True] # Early stopping to prevent overfitting and stop model training when validation score is not impr
}

mlp = MLPClassifier(random_state=42)

grid_search = GridSearchCV(
    estimator=mlp,
    param_grid=param_grid,
    cv=cv,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

grid_search.fit(X_reduced_train_scaled, y_reduced_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best cross-validation score: {grid_search.best_score_:.4f}")

```

Fitting 10 folds for each of 288 candidates, totalling 2880 fits

Best parameters: {'activation': 'relu', 'alpha': 0.01, 'early_stopping': True, 'hidden_layer_sizes': (20, 10), 'learning_rate_init': 0.01, 'max_iter': 500, 'solver': 'adam'}

Best cross-validation score: 0.9659

****With 'early stopping' and lower number of neurons in hidden layers, the MLP model converges but a lower accuracy is observed 0.9659 compared to 0.9735. We will use the mlp model with less hidden neurons that converges as the final model for MLP on the test set since the accuracy difference is not big**

```

In [35]: results_mlp = pd.DataFrame(grid_search.cv_results_)

params_results_mlp = results_mlp[['param_hidden_layer_sizes', 'param_activation', 'param_solver', 'param_alpha',
                                   'param_learning_rate_init', 'param_max_iter', 'mean_test_score', 'rank_test_score']]

params_results_mlp = params_results_mlp.sort_values(by='rank_test_score')
print(params_results_mlp.to_string())

```

	param_hidden_layer_sizes	param_activation	param_solver	param_alpha	param_learning_rate_init	param_max_iter	mean_test_sc
ore	rank_test_score						
128	(20, 10)	relu	adam	0.0100	0.010	750	0.965
930	1						
126	(20, 10)	relu	adam	0.0100	0.010	500	0.965
930	1						
130	(20, 10)	relu	adam	0.0100	0.010	1005	0.965
930	1						
78	(20, 10)	relu	adam	0.0010	0.010	500	0.964
978	4						
80	(20, 10)	relu	adam	0.0010	0.010	750	0.964
978	4						
82	(20, 10)	relu	adam	0.0010	0.010	1005	0.964
978	4						
34	(20, 10)	relu	adam	0.0001	0.010	1005	0.964
034	7						
30	(20, 10)	relu	adam	0.0001	0.010	500	0.964
034	7						
32	(20, 10)	relu	adam	0.0001	0.010	750	0.964
034	7						
142	(50, 20)	relu	adam	0.0100	0.010	1005	0.958
374	10						
140	(50, 20)	relu	adam	0.0100	0.010	750	0.958
374	10						
138	(50, 20)	relu	adam	0.0100	0.010	500	0.958
374	10						
44	(50, 20)	relu	adam	0.0001	0.010	750	0.955
508	13						
46	(50, 20)	relu	adam	0.0001	0.010	1005	0.955
508	13						
42	(50, 20)	relu	adam	0.0001	0.010	500	0.955
508	13						
92	(50, 20)	relu	adam	0.0010	0.010	750	0.953
621	16						
94	(50, 20)	relu	adam	0.0010	0.010	1005	0.953
621	16						
90	(50, 20)	relu	adam	0.0010	0.010	500	0.953
621	16						
236	(50, 20)	tanh	adam	0.0010	0.010	750	0.942
327	19						
188	(50, 20)	tanh	adam	0.0001	0.010	750	0.942

327	19						
190	(50, 20)	tanh	adam	0.0001	0.010	1005	0.942
327	19						
238	(50, 20)	tanh	adam	0.0010	0.010	1005	0.942
327	19						
186	(50, 20)	tanh	adam	0.0001	0.010	500	0.942
327	19						
234	(50, 20)	tanh	adam	0.0010	0.010	500	0.942
327	19						
282	(50, 20)	tanh	adam	0.0100	0.010	500	0.941
375	25						
286	(50, 20)	tanh	adam	0.0100	0.010	1005	0.941
375	25						
284	(50, 20)	tanh	adam	0.0100	0.010	750	0.941
375	25						
178	(20, 10)	tanh	adam	0.0001	0.010	1005	0.938
500	28						
226	(20, 10)	tanh	adam	0.0010	0.010	1005	0.938
500	28						
222	(20, 10)	tanh	adam	0.0010	0.010	500	0.938
500	28						
224	(20, 10)	tanh	adam	0.0010	0.010	750	0.938
500	28						
176	(20, 10)	tanh	adam	0.0001	0.010	750	0.938
500	28						
274	(20, 10)	tanh	adam	0.0100	0.010	1005	0.938
500	28						
174	(20, 10)	tanh	adam	0.0001	0.010	500	0.938
500	28						
272	(20, 10)	tanh	adam	0.0100	0.010	750	0.938
500	28						
270	(20, 10)	tanh	adam	0.0100	0.010	500	0.938
500	28						
22	(50,)	relu	adam	0.0001	0.010	1005	0.931
896	37						
20	(50,)	relu	adam	0.0001	0.010	750	0.931
896	37						
18	(50,)	relu	adam	0.0001	0.010	500	0.931
896	37						
114	(50,)	relu	adam	0.0100	0.010	500	0.931
896	37						

116		(50,)	relu	adam	0.0100	0.010	750	0.931
896	37							
118		(50,)	relu	adam	0.0100	0.010	1005	0.931
896	37							
70		(50,)	relu	adam	0.0010	0.010	1005	0.930
943	43							
68		(50,)	relu	adam	0.0010	0.010	750	0.930
943	43							
66		(50,)	relu	adam	0.0010	0.010	500	0.930
943	43							
250		(20,)	tanh	adam	0.0100	0.010	1005	0.927
206	46							
200		(20,)	tanh	adam	0.0010	0.010	750	0.927
206	46							
202		(20,)	tanh	adam	0.0010	0.010	1005	0.927
206	46							
248		(20,)	tanh	adam	0.0100	0.010	750	0.927
206	46							
246		(20,)	tanh	adam	0.0100	0.010	500	0.927
206	46							
198		(20,)	tanh	adam	0.0010	0.010	500	0.927
206	46							
210		(50,)	tanh	adam	0.0010	0.010	500	0.926
226	52							
260		(50,)	tanh	adam	0.0100	0.010	750	0.926
226	52							
258		(50,)	tanh	adam	0.0100	0.010	500	0.926
226	52							
162		(50,)	tanh	adam	0.0001	0.010	500	0.926
226	52							
164		(50,)	tanh	adam	0.0001	0.010	750	0.926
226	52							
166		(50,)	tanh	adam	0.0001	0.010	1005	0.926
226	52							
214		(50,)	tanh	adam	0.0010	0.010	1005	0.926
226	52							
212		(50,)	tanh	adam	0.0010	0.010	750	0.926
226	52							
262		(50,)	tanh	adam	0.0100	0.010	1005	0.926
226	52							
154		(20,)	tanh	adam	0.0001	0.010	1005	0.922

489	61							
150	(20,)	tanh	adam	0.0001	0.010	500	0.922	
489	61							
152	(20,)	tanh	adam	0.0001	0.010	750	0.922	
489	61							
58	(20,)	relu	adam	0.0010	0.010	1005	0.917	
691	64							
56	(20,)	relu	adam	0.0010	0.010	750	0.917	
691	64							
6	(20,)	relu	adam	0.0001	0.010	500	0.917	
691	64							
8	(20,)	relu	adam	0.0001	0.010	750	0.917	
691	64							
54	(20,)	relu	adam	0.0010	0.010	500	0.917	
691	64							
10	(20,)	relu	adam	0.0001	0.010	1005	0.917	
691	64							
102	(20,)	relu	adam	0.0100	0.010	500	0.914	
852	70							
104	(20,)	relu	adam	0.0100	0.010	750	0.914	
852	70							
106	(20,)	relu	adam	0.0100	0.010	1005	0.914	
852	70							
191	(50, 20)	tanh	sgd	0.0001	0.010	1005	0.897	
772	73							
239	(50, 20)	tanh	sgd	0.0010	0.010	1005	0.897	
772	73							
235	(50, 20)	tanh	sgd	0.0010	0.010	500	0.897	
772	73							
189	(50, 20)	tanh	sgd	0.0001	0.010	750	0.897	
772	73							
283	(50, 20)	tanh	sgd	0.0100	0.010	500	0.897	
772	73							
287	(50, 20)	tanh	sgd	0.0100	0.010	1005	0.897	
772	73							
187	(50, 20)	tanh	sgd	0.0001	0.010	500	0.897	
772	73							
285	(50, 20)	tanh	sgd	0.0100	0.010	750	0.897	
772	73							
237	(50, 20)	tanh	sgd	0.0010	0.010	750	0.897	
772	73							

184	(50, 20)	tanh	adam	0.0001	0.001	1005	0.876
909	82						
278	(50, 20)	tanh	adam	0.0100	0.001	750	0.876
909	82						
230	(50, 20)	tanh	adam	0.0010	0.001	750	0.876
909	82						
232	(50, 20)	tanh	adam	0.0010	0.001	1005	0.876
909	82						
182	(50, 20)	tanh	adam	0.0001	0.001	750	0.876
909	82						
228	(50, 20)	tanh	adam	0.0010	0.001	500	0.876
909	82						
180	(50, 20)	tanh	adam	0.0001	0.001	500	0.876
909	82						
280	(50, 20)	tanh	adam	0.0100	0.001	1005	0.876
909	82						
276	(50, 20)	tanh	adam	0.0100	0.001	500	0.876
909	82						
227	(20, 10)	tanh	sgd	0.0010	0.010	1005	0.873
100	91						
225	(20, 10)	tanh	sgd	0.0010	0.010	750	0.873
100	91						
271	(20, 10)	tanh	sgd	0.0100	0.010	500	0.873
100	91						
177	(20, 10)	tanh	sgd	0.0001	0.010	750	0.873
100	91						
223	(20, 10)	tanh	sgd	0.0010	0.010	500	0.873
100	91						
179	(20, 10)	tanh	sgd	0.0001	0.010	1005	0.873
100	91						
275	(20, 10)	tanh	sgd	0.0100	0.010	1005	0.873
100	91						
273	(20, 10)	tanh	sgd	0.0100	0.010	750	0.873
100	91						
175	(20, 10)	tanh	sgd	0.0001	0.010	500	0.873
100	91						
88	(50, 20)	relu	adam	0.0010	0.001	1005	0.868
392	100						
36	(50, 20)	relu	adam	0.0001	0.001	500	0.868
392	100						
86	(50, 20)	relu	adam	0.0010	0.001	750	0.868

392	100						
84	(50, 20)	relu	adam	0.0010	0.001	500	0.868
392	100						
38	(50, 20)	relu	adam	0.0001	0.001	750	0.868
392	100						
40	(50, 20)	relu	adam	0.0001	0.001	1005	0.868
392	100						
31	(20, 10)	relu	sgd	0.0001	0.010	500	0.865
544	106						
33	(20, 10)	relu	sgd	0.0001	0.010	750	0.865
544	106						
81	(20, 10)	relu	sgd	0.0010	0.010	750	0.865
544	106						
131	(20, 10)	relu	sgd	0.0100	0.010	1005	0.865
544	106						
129	(20, 10)	relu	sgd	0.0100	0.010	750	0.865
544	106						
35	(20, 10)	relu	sgd	0.0001	0.010	1005	0.865
544	106						
79	(20, 10)	relu	sgd	0.0010	0.010	500	0.865
544	106						
83	(20, 10)	relu	sgd	0.0010	0.010	1005	0.865
544	106						
127	(20, 10)	relu	sgd	0.0100	0.010	500	0.865
544	106						
132	(50, 20)	relu	adam	0.0100	0.001	500	0.864
591	115						
136	(50, 20)	relu	adam	0.0100	0.001	1005	0.864
591	115						
134	(50, 20)	relu	adam	0.0100	0.001	750	0.864
591	115						
28	(20, 10)	relu	adam	0.0001	0.001	1005	0.864
546	118						
26	(20, 10)	relu	adam	0.0001	0.001	750	0.864
546	118						
24	(20, 10)	relu	adam	0.0001	0.001	500	0.864
546	118						
74	(20, 10)	relu	adam	0.0010	0.001	750	0.864
546	118						
72	(20, 10)	relu	adam	0.0010	0.001	500	0.864
546	118						

76	(20, 10)	relu	adam	0.0010	0.001	1005	0.864
546	118						
167	(50,)	tanh	sgd	0.0001	0.010	1005	0.863
648	124						
165	(50,)	tanh	sgd	0.0001	0.010	750	0.863
648	124						
163	(50,)	tanh	sgd	0.0001	0.010	500	0.863
648	124						
211	(50,)	tanh	sgd	0.0010	0.010	500	0.863
648	124						
213	(50,)	tanh	sgd	0.0010	0.010	750	0.863
648	124						
215	(50,)	tanh	sgd	0.0010	0.010	1005	0.863
648	124						
263	(50,)	tanh	sgd	0.0100	0.010	1005	0.863
648	124						
259	(50,)	tanh	sgd	0.0100	0.010	500	0.863
648	124						
261	(50,)	tanh	sgd	0.0100	0.010	750	0.863
648	124						
120	(20, 10)	relu	adam	0.0100	0.001	500	0.863
603	133						
124	(20, 10)	relu	adam	0.0100	0.001	1005	0.863
603	133						
122	(20, 10)	relu	adam	0.0100	0.001	750	0.863
603	133						
181	(50, 20)	tanh	sgd	0.0001	0.001	500	0.857
951	136						
279	(50, 20)	tanh	sgd	0.0100	0.001	750	0.857
951	136						
185	(50, 20)	tanh	sgd	0.0001	0.001	1005	0.857
951	136						
229	(50, 20)	tanh	sgd	0.0010	0.001	500	0.857
951	136						
233	(50, 20)	tanh	sgd	0.0010	0.001	1005	0.857
951	136						
231	(50, 20)	tanh	sgd	0.0010	0.001	750	0.857
951	136						
277	(50, 20)	tanh	sgd	0.0100	0.001	500	0.857
951	136						
281	(50, 20)	tanh	sgd	0.0100	0.001	1005	0.857

951	136						
183	(50, 20)	tanh	sgd	0.0001	0.001	750	0.857
951	136						
95	(50, 20)	relu	sgd	0.0010	0.010	1005	0.856
029	145						
143	(50, 20)	relu	sgd	0.0100	0.010	1005	0.856
029	145						
139	(50, 20)	relu	sgd	0.0100	0.010	500	0.856
029	145						
141	(50, 20)	relu	sgd	0.0100	0.010	750	0.856
029	145						
91	(50, 20)	relu	sgd	0.0010	0.010	500	0.856
029	145						
47	(50, 20)	relu	sgd	0.0001	0.010	1005	0.856
029	145						
43	(50, 20)	relu	sgd	0.0001	0.010	500	0.856
029	145						
45	(50, 20)	relu	sgd	0.0001	0.010	750	0.856
029	145						
93	(50, 20)	relu	sgd	0.0010	0.010	750	0.856
029	145						
67	(50,)	relu	sgd	0.0010	0.010	500	0.852
273	154						
69	(50,)	relu	sgd	0.0010	0.010	750	0.852
273	154						
23	(50,)	relu	sgd	0.0001	0.010	1005	0.852
273	154						
21	(50,)	relu	sgd	0.0001	0.010	750	0.852
273	154						
19	(50,)	relu	sgd	0.0001	0.010	500	0.852
273	154						
71	(50,)	relu	sgd	0.0010	0.010	1005	0.852
273	154						
119	(50,)	relu	sgd	0.0100	0.010	1005	0.852
273	154						
117	(50,)	relu	sgd	0.0100	0.010	750	0.852
273	154						
115	(50,)	relu	sgd	0.0100	0.010	500	0.852
273	154						
266	(20, 10)	tanh	adam	0.0100	0.001	750	0.851
330	163						

168	(20, 10)	tanh	adam	0.0001	0.001	500	0.851
330	163						
170	(20, 10)	tanh	adam	0.0001	0.001	750	0.851
330	163						
216	(20, 10)	tanh	adam	0.0010	0.001	500	0.851
330	163						
218	(20, 10)	tanh	adam	0.0010	0.001	750	0.851
330	163						
220	(20, 10)	tanh	adam	0.0010	0.001	1005	0.851
330	163						
264	(20, 10)	tanh	adam	0.0100	0.001	500	0.851
330	163						
268	(20, 10)	tanh	adam	0.0100	0.001	1005	0.851
330	163						
172	(20, 10)	tanh	adam	0.0001	0.001	1005	0.851
330	163						
219	(20, 10)	tanh	sgd	0.0010	0.001	750	0.842
749	172						
221	(20, 10)	tanh	sgd	0.0010	0.001	1005	0.842
749	172						
267	(20, 10)	tanh	sgd	0.0100	0.001	750	0.842
749	172						
265	(20, 10)	tanh	sgd	0.0100	0.001	500	0.842
749	172						
269	(20, 10)	tanh	sgd	0.0100	0.001	1005	0.842
749	172						
169	(20, 10)	tanh	sgd	0.0001	0.001	500	0.842
749	172						
173	(20, 10)	tanh	sgd	0.0001	0.001	1005	0.842
749	172						
171	(20, 10)	tanh	sgd	0.0001	0.001	750	0.842
749	172						
217	(20, 10)	tanh	sgd	0.0010	0.001	500	0.842
749	172						
247	(20,)	tanh	sgd	0.0100	0.010	500	0.839
937	181						
153	(20,)	tanh	sgd	0.0001	0.010	750	0.839
937	181						
199	(20,)	tanh	sgd	0.0010	0.010	500	0.839
937	181						
203	(20,)	tanh	sgd	0.0010	0.010	1005	0.839

937	181						
151	(20,)	tanh	sgd	0.0001	0.010	500	0.839
937	181						
155	(20,)	tanh	sgd	0.0001	0.010	1005	0.839
937	181						
201	(20,)	tanh	sgd	0.0010	0.010	750	0.839
937	181						
249	(20,)	tanh	sgd	0.0100	0.010	750	0.839
937	181						
251	(20,)	tanh	sgd	0.0100	0.010	1005	0.839
937	181						
107	(20,)	relu	sgd	0.0100	0.010	1005	0.839
937	190						
7	(20,)	relu	sgd	0.0001	0.010	500	0.839
937	190						
57	(20,)	relu	sgd	0.0010	0.010	750	0.839
937	190						
55	(20,)	relu	sgd	0.0010	0.010	500	0.839
937	190						
59	(20,)	relu	sgd	0.0010	0.010	1005	0.839
937	190						
103	(20,)	relu	sgd	0.0100	0.010	500	0.839
937	190						
11	(20,)	relu	sgd	0.0001	0.010	1005	0.839
937	190						
9	(20,)	relu	sgd	0.0001	0.010	750	0.839
937	190						
105	(20,)	relu	sgd	0.0100	0.010	750	0.839
937	190						
206	(50,)	tanh	adam	0.0010	0.001	750	0.838
994	199						
252	(50,)	tanh	adam	0.0100	0.001	500	0.838
994	199						
254	(50,)	tanh	adam	0.0100	0.001	750	0.838
994	199						
256	(50,)	tanh	adam	0.0100	0.001	1005	0.838
994	199						
204	(50,)	tanh	adam	0.0010	0.001	500	0.838
994	199						
156	(50,)	tanh	adam	0.0001	0.001	500	0.838
994	199						

208		(50,)	tanh	adam	0.0010	0.001	1005	0.838
994	199							
158		(50,)	tanh	adam	0.0001	0.001	750	0.838
994	199							
160		(50,)	tanh	adam	0.0001	0.001	1005	0.838
994	199							
60		(50,)	relu	adam	0.0010	0.001	500	0.829
524	208							
62		(50,)	relu	adam	0.0010	0.001	750	0.829
524	208							
14		(50,)	relu	adam	0.0001	0.001	750	0.829
524	208							
112		(50,)	relu	adam	0.0100	0.001	1005	0.829
524	208							
110		(50,)	relu	adam	0.0100	0.001	750	0.829
524	208							
108		(50,)	relu	adam	0.0100	0.001	500	0.829
524	208							
12		(50,)	relu	adam	0.0001	0.001	500	0.829
524	208							
16		(50,)	relu	adam	0.0001	0.001	1005	0.829
524	208							
64		(50,)	relu	adam	0.0010	0.001	1005	0.829
524	208							
144		(20,)	tanh	adam	0.0001	0.001	500	0.826
658	217							
196		(20,)	tanh	adam	0.0010	0.001	1005	0.826
658	217							
148		(20,)	tanh	adam	0.0001	0.001	1005	0.826
658	217							
242		(20,)	tanh	adam	0.0100	0.001	750	0.826
658	217							
240		(20,)	tanh	adam	0.0100	0.001	500	0.826
658	217							
244		(20,)	tanh	adam	0.0100	0.001	1005	0.826
658	217							
194		(20,)	tanh	adam	0.0010	0.001	750	0.826
658	217							
192		(20,)	tanh	adam	0.0010	0.001	500	0.826
658	217							
146		(20,)	tanh	adam	0.0001	0.001	750	0.826

658	217							
253	226	(50,)	tanh	sgd	0.0100	0.001	500	0.821
860	226	(50,)	tanh	sgd	0.0100	0.001	750	0.821
255	226	(50,)	tanh	sgd	0.0001	0.001	750	0.821
860	226	(50,)	tanh	sgd	0.0001	0.001	500	0.821
159	226	(50,)	tanh	sgd	0.0001	0.001	1005	0.821
860	226	(50,)	tanh	sgd	0.0010	0.001	500	0.821
157	226	(50,)	tanh	sgd	0.0010	0.001	750	0.821
860	226	(50,)	tanh	sgd	0.0100	0.001	1005	0.821
161	226	(50,)	tanh	sgd	0.0010	0.001	1005	0.821
860	226	(50,)	tanh	sgd	0.0010	0.001	750	0.821
205	226	(50,)	tanh	sgd	0.0100	0.001	1005	0.821
860	226	(50,)	tanh	sgd	0.0010	0.001	500	0.820
207	235	(20,)	relu	adam	0.0100	0.001	1005	0.820
860	235	(20,)	relu	adam	0.0100	0.001	750	0.820
257	235	(20,)	relu	adam	0.0010	0.001	500	0.820
860	235	(20,)	relu	adam	0.0010	0.001	1005	0.820
209	235	(20,)	relu	adam	0.0010	0.001	500	0.820
860	235	(20,)	relu	adam	0.0001	0.001	1005	0.820
50	235	(20,)	relu	adam	0.0001	0.001	750	0.820
997	235	(20,)	relu	adam	0.0010	0.001	500	0.793
100	244	(50, 20)	relu	sgd	0.0010	0.001	500	0.793
997	244	(50, 20)	relu	sgd	0.0010	0.001	500	0.793

87	(50, 20)	relu	sgd	0.0010	0.001	750	0.793
450	244						
39	(50, 20)	relu	sgd	0.0001	0.001	750	0.793
450	244						
41	(50, 20)	relu	sgd	0.0001	0.001	1005	0.793
450	244						
135	(50, 20)	relu	sgd	0.0100	0.001	750	0.793
450	244						
137	(50, 20)	relu	sgd	0.0100	0.001	1005	0.793
450	244						
133	(50, 20)	relu	sgd	0.0100	0.001	500	0.793
450	244						
37	(50, 20)	relu	sgd	0.0001	0.001	500	0.793
450	244						
65	(50,)	relu	sgd	0.0010	0.001	1005	0.790
575	253						
15	(50,)	relu	sgd	0.0001	0.001	750	0.790
575	253						
109	(50,)	relu	sgd	0.0100	0.001	500	0.790
575	253						
61	(50,)	relu	sgd	0.0010	0.001	500	0.790
575	253						
63	(50,)	relu	sgd	0.0010	0.001	750	0.790
575	253						
13	(50,)	relu	sgd	0.0001	0.001	500	0.790
575	253						
113	(50,)	relu	sgd	0.0100	0.001	1005	0.790
575	253						
17	(50,)	relu	sgd	0.0001	0.001	1005	0.790
575	253						
111	(50,)	relu	sgd	0.0100	0.001	750	0.790
575	253						
245	(20,)	tanh	sgd	0.0100	0.001	1005	0.768
562	262						
193	(20,)	tanh	sgd	0.0010	0.001	500	0.768
562	262						
145	(20,)	tanh	sgd	0.0001	0.001	500	0.768
562	262						
241	(20,)	tanh	sgd	0.0100	0.001	500	0.768
562	262						
195	(20,)	tanh	sgd	0.0010	0.001	750	0.768

562	262							
197	(20,)	tanh	sgd	0.0010	0.001	1005	0.768	
562	262							
149	(20,)	tanh	sgd	0.0001	0.001	1005	0.768	
562	262							
147	(20,)	tanh	sgd	0.0001	0.001	750	0.768	
562	262							
243	(20,)	tanh	sgd	0.0100	0.001	750	0.768	
562	262							
29	(20, 10)	relu	sgd	0.0001	0.001	1005	0.761	
195	271							
25	(20, 10)	relu	sgd	0.0001	0.001	500	0.761	
195	271							
77	(20, 10)	relu	sgd	0.0010	0.001	1005	0.761	
195	271							
75	(20, 10)	relu	sgd	0.0010	0.001	750	0.761	
195	271							
27	(20, 10)	relu	sgd	0.0001	0.001	750	0.761	
195	271							
123	(20, 10)	relu	sgd	0.0100	0.001	750	0.761	
195	271							
73	(20, 10)	relu	sgd	0.0010	0.001	500	0.761	
195	271							
121	(20, 10)	relu	sgd	0.0100	0.001	500	0.761	
195	271							
125	(20, 10)	relu	sgd	0.0100	0.001	1005	0.761	
195	271							
3	(20,)	relu	sgd	0.0001	0.001	750	0.739	
479	280							
99	(20,)	relu	sgd	0.0100	0.001	750	0.739	
479	280							
1	(20,)	relu	sgd	0.0001	0.001	500	0.739	
479	280							
5	(20,)	relu	sgd	0.0001	0.001	1005	0.739	
479	280							
51	(20,)	relu	sgd	0.0010	0.001	750	0.739	
479	280							
49	(20,)	relu	sgd	0.0010	0.001	500	0.739	
479	280							
53	(20,)	relu	sgd	0.0010	0.001	1005	0.739	
479	280							

101	(20,)	relu	sgd	0.0100	0.001	1005	0.739
479	280						
97	(20,)	relu	sgd	0.0100	0.001	500	0.739
479	280						

Applying best models on the Test Set

SVM Best Model

```
In [36]: # Test the best model on the test set
# From GridSearch: 'C': 100, 'gamma': 0.1, 'kernel': 'rbf'
best_svm = grid_search.best_estimator_
test_score = best_svm.score(X_reduced_test_scaled, y_reduced_test)
print(f"SVM Test accuracy with best parameters: {test_score:.4f}")
```

SVM Test accuracy with best parameters: 0.9470

```
In [37]: # Predictions of Test set
y_pred = best_svm.predict(X_reduced_test_scaled)
```

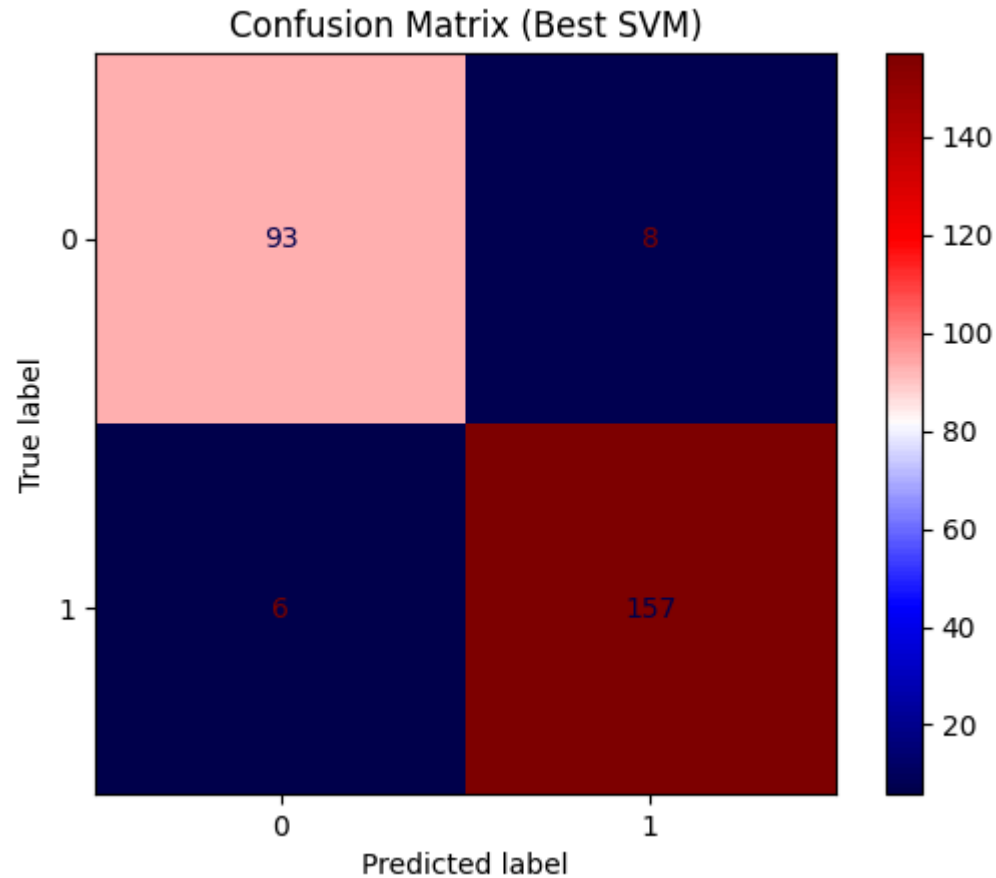
```
In [38]: # SVM Classification Report
print("\nClassification Report (SVM):")
print(classification_report(y_reduced_test, y_pred))
```

Classification Report (SVM):

	precision	recall	f1-score	support
0	0.94	0.92	0.93	101
1	0.95	0.96	0.96	163
accuracy			0.95	264
macro avg	0.95	0.94	0.94	264
weighted avg	0.95	0.95	0.95	264

```
In [39]: # SVM Confusion Matrix
plt.figure(figsize=(5, 4))
cm_bestSVM = ConfusionMatrixDisplay.from_predictions(y_reduced_test, y_pred, cmap='seismic')
plt.title("Confusion Matrix (Best SVM)")
plt.show()
```

<Figure size 500x400 with 0 Axes>



```
In [40]: print(best_svm.get_params())
```

```
{'activation': 'relu', 'alpha': 0.01, 'batch_size': 'auto', 'beta_1': 0.9, 'beta_2': 0.999, 'early_stopping': True, 'epsilon': 1e-08, 'hidden_layer_sizes': (20, 10), 'learning_rate': 'constant', 'learning_rate_init': 0.01, 'max_fun': 15000, 'max_iter': 500, 'momentum': 0.9, 'n_iter_no_change': 10, 'nesterovs_momentum': True, 'power_t': 0.5, 'random_state': 42, 'shuffle': True, 'solver': 'adam', 'tol': 0.0001, 'validation_fraction': 0.1, 'verbose': False, 'warm_start': False}
```

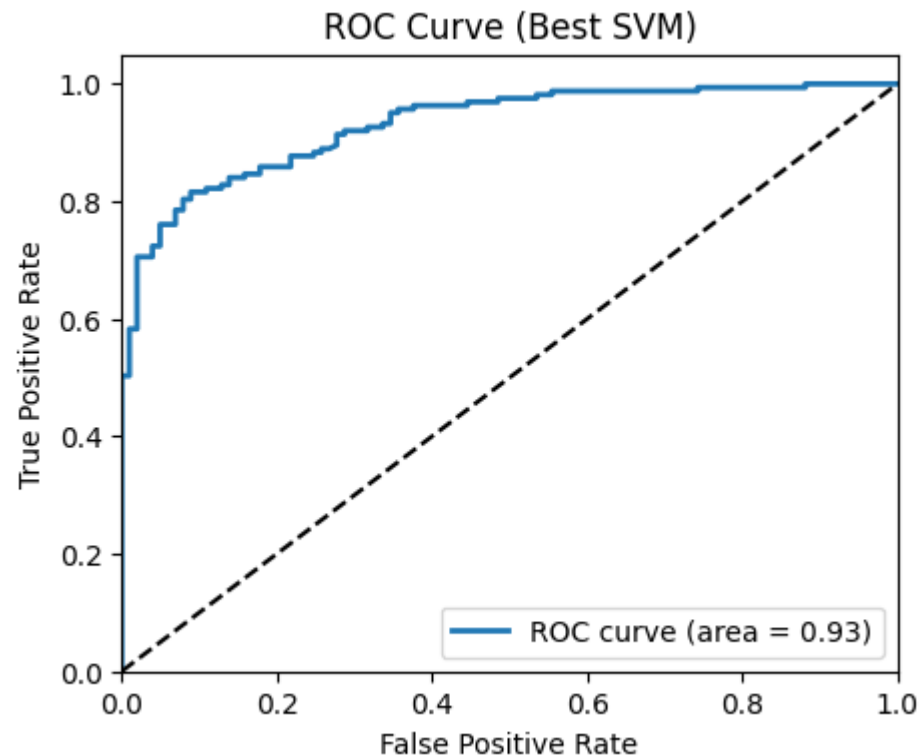
```
In [41]: # For ROC AUC Score with SVC
```

```
svm_roc = SVC(probability=True, random_state=42) # Basic SVC with probability enabled
svm_roc.fit(X_reduced_train_scaled, y_reduced_train)
y_prob = svm_roc.predict_proba(X_reduced_test_scaled)
```

```
roc_auc = roc_auc_score(y_reduced_test, y_prob[:, 1]) # Calc roc-auc score
print(f"ROC-AUC Score: {roc_auc:.4f}")
```

ROC-AUC Score: 0.9308

```
In [42]: # Plot ROC curve
fpr, tpr, thresholds = roc_curve(y_reduced_test, y_prob[:, 1])
plt.figure(figsize=(5, 4))
plt.plot(fpr, tpr, lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve (Best SVM)')
plt.legend()
plt.show()
```



MLP Best Model

```
In [43]: # Test the best MLP model
# From GridSearch: 'activation': 'tanh', 'alpha': 0.0001, 'hidden_layer_sizes': (50, 50), 'learning_rate_init': 0.001, 'max_it
best_mlp = grid_search.best_estimator_
test_score = best_mlp.score(X_reduced_test_scaled, y_reduced_test)
print(f"Test accuracy with best parameters: {test_score:.4f}")
```

Test accuracy with best parameters: 0.9470

```
In [44]: # Predictions of Test set
y_pred_mlp = best_mlp.predict(X_reduced_test_scaled)
```

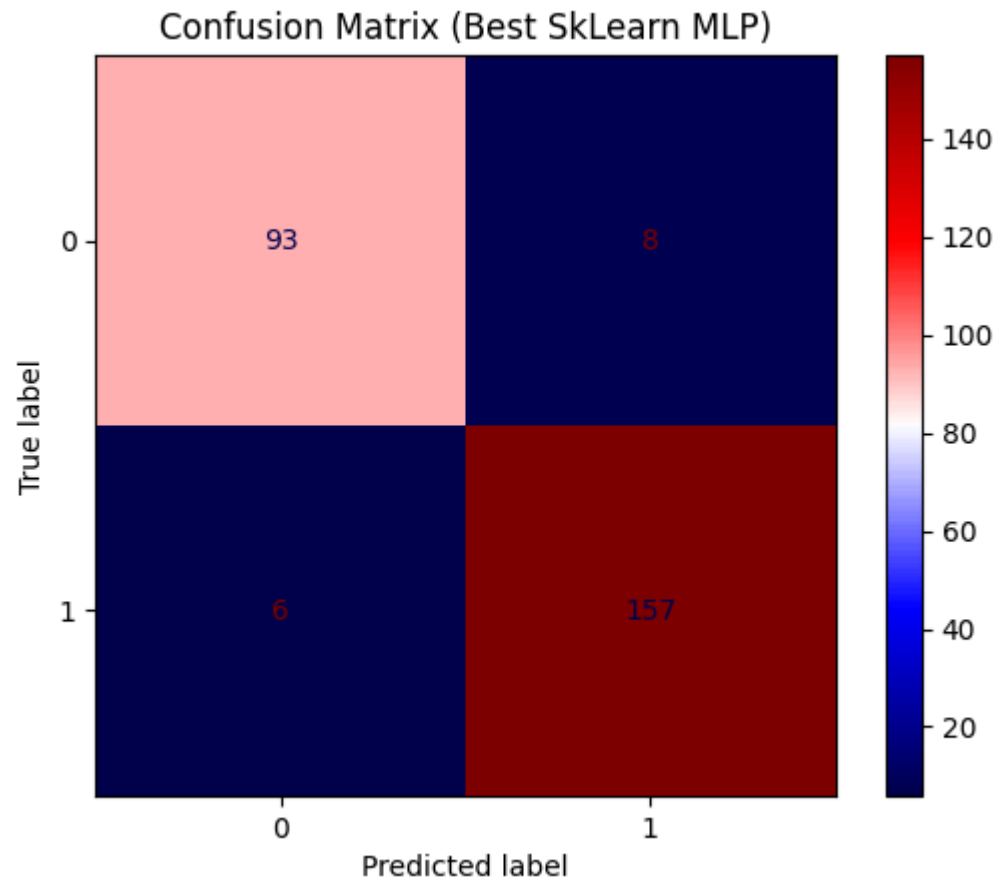
```
In [45]: # MLP Classification Report
print("\nClassification Report (MLP):")
print(classification_report(y_reduced_test, y_pred_mlp))
```

Classification Report (MLP):

	precision	recall	f1-score	support
0	0.94	0.92	0.93	101
1	0.95	0.96	0.96	163
accuracy			0.95	264
macro avg	0.95	0.94	0.94	264
weighted avg	0.95	0.95	0.95	264

```
In [46]: # MLP Confusion Matrix
plt.figure(figsize=(5, 4))
cm_bestmlp = ConfusionMatrixDisplay.from_predictions(y_reduced_test, y_pred_mlp, cmap='seismic')
plt.title("Confusion Matrix (Best SkLearn MLP)")
plt.show()
```

<Figure size 500x400 with 0 Axes>



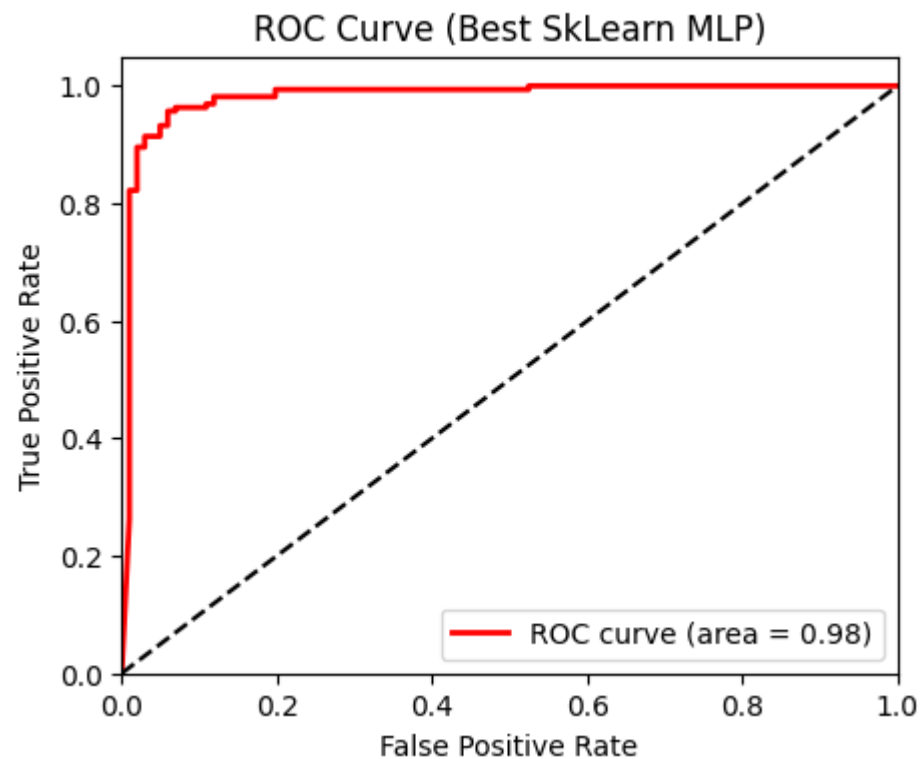
```
In [47]: # ROC AUC Score MLP
mlp_roc = MLPClassifier(**best_mlp.get_params())
mlp_roc.fit(X_reduced_train_scaled, y_reduced_train)
y_prob = mlp_roc.predict_proba(X_reduced_test_scaled)

roc_auc = roc_auc_score(y_reduced_test, y_prob[:, 1]) # calc roc auc score
print(f"ROC-AUC Score: {roc_auc:.4f}")
```

ROC-AUC Score: 0.9806

```
In [48]: # Plot ROC curve
fpr, tpr, thresholds = roc_curve(y_reduced_test, y_prob[:, 1])
```

```
# Create the plot
plt.figure(figsize=(5, 4))
plt.plot(fpr, tpr, color='red', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve (Best SkLearn MLP)')
plt.legend()
plt.show()
```



Test set for Base MLP to obtain Confusion Matrix for Base MLP

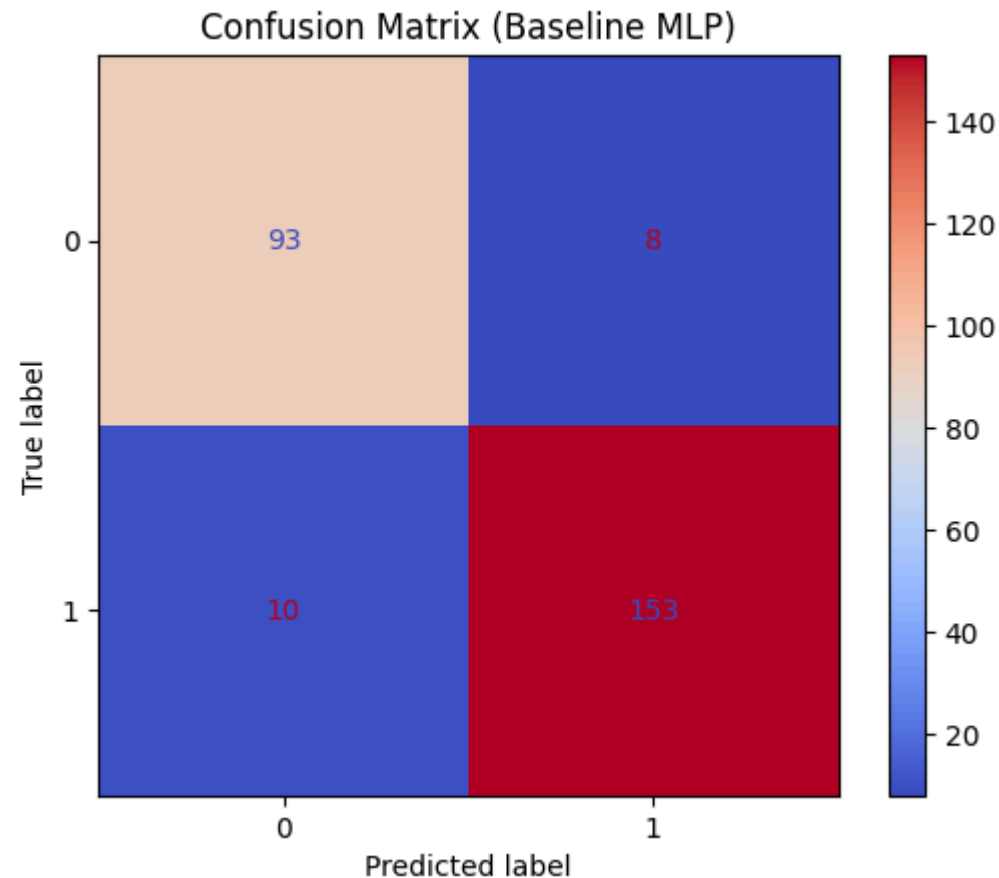
```
In [49]: mlp_basemodel.fit(X2_train_scaled, y2_train)
```

```
# Make predictions on the test set
y2_pred_mlp = mlp_basemodel.predict(X2_test_scaled)

# Create the confusion matrix display with a name
plt.figure(figsize=(5, 4))
cm_basemlp = ConfusionMatrixDisplay.from_predictions(y2_test, y2_pred_mlp, cmap='coolwarm')
plt.title("Confusion Matrix (Baseline MLP)")
plt.show()
```

c:\Users\sajja\anaconda3\envs\neuralcomp\env\Lib\site-packages\sklearn\normal_network_multilayer_perceptron.py:691: Convergence Warning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.

warnings.warn(
<Figure size 500x400 with 0 Axes>



PyTorch Implementation (Base MLP)

The implementation of the base MLP model will be similar to the base SKLearn MLP model in terms of hyperparameters and features being used. We implement the model using both set of features to monitor performance.

```
In [50]: # Initialize the layers in MLP model
mlp = MLPClassifier(random_state=42)
class BaseMLP(nn.Module):
    def __init__(self, input_dim, hidden_dim=100, output_dim=1): # The dimension for hidden neuron and output are set in a way
        super(BaseMLP, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, output_dim)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)

        return x

In [51]: # Instantiating the models (weights and biases are initialized randomly)
input_dim1 = X1_train_scaled.shape[1]
input_dim2 = X2_train_scaled.shape[1]

In [52]: basemodel_X1 = BaseMLP(input_dim1)
basemodel_X2 = BaseMLP(input_dim2)

In [53]: # Setting the optimizers and loss function
optimizer_X1 = optim.Adam(basemodel_X1.parameters(), lr=0.001, weight_decay=0.001)
optimizer_X2 = optim.Adam(basemodel_X2.parameters(), lr=0.001, weight_decay=0.001)
criterion = nn.BCEWithLogitsLoss() # Binary Cross-Entropy Loss

In [54]: # Creating tensor datasets and data loaders
train_dataset_X1 = TensorDataset(torch.FloatTensor(X1_train_scaled), torch.FloatTensor(y1_train.values).view(-1, 1))
train_dataset_X2 = TensorDataset(torch.FloatTensor(X2_train_scaled), torch.FloatTensor(y2_train.values).view(-1, 1))

batch_size = 32 # or 64 could also work (These two are common batch sizes)
```

```
train_loader_X1 = DataLoader(train_dataset_X1, batch_size=batch_size, shuffle=True)
train_loader_X2 = DataLoader(train_dataset_X2, batch_size=batch_size, shuffle=True)
```

```
In [55]: def kfold_cv(dataset, basemodel, criterion, optimizer, feature_name, n_splits=10, max_iter=100, batch_size=batch_size, patience=10):

    # Get tensors from dataset
    X_tensor = dataset.tensors[0]
    y_tensor = dataset.tensors[1]
    # Converting to numpy for KFold splitting (since SKlearn KFold doesn't work with PyTorch tensors)
    X_np = X_tensor.numpy()
    y_np = y_tensor.numpy()

    # Initialize KFold
    kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)

    # To store evaluation metrics
    fold accuracies = []
    fold losses = []

    # Loop through folds
    for fold, (train_idx, val_idx) in enumerate(kf.split(X_np)):
        print(f"\nTraining fold {fold+1}/{n_splits}:")

        # Create fold-specific datasets
        X_train_fold = X_tensor[train_idx]
        y_train_fold = y_tensor[train_idx]
        X_val_fold = X_tensor[val_idx]
        y_val_fold = y_tensor[val_idx]

        # Creating fold-specific dataloader
        train_fold_dataset = torch.utils.data.TensorDataset(X_train_fold, y_train_fold)
        train_fold_loader = torch.utils.data.DataLoader(train_fold_dataset, batch_size=batch_size, shuffle=True)

        # Reset model weights for each fold
        for layer in basemodel.children():
            if hasattr(layer, 'reset_parameters'):
                layer.reset_parameters()

        # Create a fresh optimizer for this fold
        fold_optimizer = torch.optim.Adam(
```

```
basemodel.parameters(),
lr=optimizer.param_groups[0]['lr'],
weight_decay=optimizer.param_groups[0]['weight_decay']
)

# Early stopping
best_val_loss = float('inf')
early_stop_counter = 0

# Training loop
for epoch in range(max_iter):
    # Set to training mode
    basemodel.train()

    # Track metrics
    running_loss = 0.0

    # Batch training
    for inputs, targets in train_fold_loader:
        # Forward pass from inputs
        outputs = basemodel(inputs)
        loss = criterion(outputs, targets)

        # Backward and optimisation
        fold_optimizer.zero_grad()
        loss.backward()
        fold_optimizer.step()

        running_loss += loss.item()

    # Average training loss
    epoch_loss = running_loss / len(train_fold_loader)

    # Validation
    basemodel.eval()
    with torch.no_grad():
        val_outputs = basemodel(X_val_fold)
        val_loss = criterion(val_outputs, y_val_fold).item()

    # Compute validation accuracy
    val_preds = (torch.sigmoid(val_outputs) > 0.5).float()
```

```

        val_accuracy = (val_preds == y_val_fold).sum().item() / len(y_val_fold)

        # Print progress every 10 epochs and at last epoch
        if (epoch + 1) % 10 == 0 or epoch == max_iter - 1:
            print(f"Epoch {epoch+1}/{max_iter}, Train Loss: {epoch_loss:.4f}, Val Loss: {val_loss:.4f}, Val Acc: {val_accu

        # Early stopping check
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            early_stop_counter = 0
        else:
            early_stop_counter += 1

        if early_stop_counter >= patience: # Output early stopping message
            print(f'Early stopping triggered at epoch {epoch+1}')
            break

    basemodel.eval() # Final validation
    with torch.no_grad():
        val_outputs = basemodel(X_val_fold)
        val_preds = (torch.sigmoid(val_outputs) > 0.5).float()
        val_accuracy = (val_preds == y_val_fold).sum().item() / len(y_val_fold)

    # Store results
    fold accuracies.append(val_accuracy)
    fold_losses.append(best_val_loss)

    print(f"Fold {fold+1} - Validation Accuracy: {val_accuracy:.4f}")

cv_accuracy = np.mean(fold accuracies) # Calculate overall metrics

print(f"\n{n_splits}-Fold CV Results:")
print(f"Mean Accuracy: {cv_accuracy:.4f}")

return cv_accuracy, fold accuracies

```

```

In [56]: # Run cross-validation for the Systolic BP model (X1)
print(" Evaluating Systolic BP Model (X1 features)")
X1_cv_acc, X1_fold_accs = kfold_cv(
    dataset=train_dataset_X1,

```

```
    basemodel=basemodel_X1,  
    criterion=criterion,  
    optimizer=optimizer_X1,  
    feature_name="MLP (Systolic BP)"  
)  
  
# Run cross-validation for the Pulse Pressure model (X2)  
print("Evaluating Pulse Pressure Model (X2 features)")  
X2_cv_acc, X2_fold_accs = kfold_cv(  
    dataset=train_dataset_X2,  
    basemodel=basemodel_X2,  
    criterion=criterion,  
    optimizer=optimizer_X2,  
    feature_name="MLP (Pulse Pressure)"  
)
```

Evaluating Systolic BP Model (X1 features)

Training fold 1/10:

Epoch 10/100, Train Loss: 0.3226, Val Loss: 0.4099, Val Acc: 0.7736
Epoch 20/100, Train Loss: 0.2472, Val Loss: 0.3205, Val Acc: 0.8679
Epoch 30/100, Train Loss: 0.2039, Val Loss: 0.2716, Val Acc: 0.8962
Epoch 40/100, Train Loss: 0.1773, Val Loss: 0.2443, Val Acc: 0.9057
Epoch 50/100, Train Loss: 0.1607, Val Loss: 0.2216, Val Acc: 0.9245
Epoch 60/100, Train Loss: 0.1483, Val Loss: 0.1972, Val Acc: 0.9245
Epoch 70/100, Train Loss: 0.1370, Val Loss: 0.1675, Val Acc: 0.9528
Epoch 80/100, Train Loss: 0.1272, Val Loss: 0.1731, Val Acc: 0.9528
Epoch 90/100, Train Loss: 0.1207, Val Loss: 0.1707, Val Acc: 0.9528
Early stopping triggered at epoch 94
Fold 1 - Validation Accuracy: 0.9528

Training fold 2/10:

Epoch 10/100, Train Loss: 0.2941, Val Loss: 0.4054, Val Acc: 0.8302
Epoch 20/100, Train Loss: 0.2100, Val Loss: 0.4104, Val Acc: 0.8868
Early stopping triggered at epoch 22
Fold 2 - Validation Accuracy: 0.9057

Training fold 3/10:

Epoch 10/100, Train Loss: 0.3269, Val Loss: 0.3347, Val Acc: 0.8774
Epoch 20/100, Train Loss: 0.2572, Val Loss: 0.2574, Val Acc: 0.8962
Epoch 30/100, Train Loss: 0.2094, Val Loss: 0.2082, Val Acc: 0.9151
Epoch 40/100, Train Loss: 0.1799, Val Loss: 0.1750, Val Acc: 0.9434
Epoch 50/100, Train Loss: 0.1609, Val Loss: 0.1563, Val Acc: 0.9434
Epoch 60/100, Train Loss: 0.1435, Val Loss: 0.1445, Val Acc: 0.9623
Epoch 70/100, Train Loss: 0.1348, Val Loss: 0.1391, Val Acc: 0.9623
Epoch 80/100, Train Loss: 0.1262, Val Loss: 0.1326, Val Acc: 0.9623
Epoch 90/100, Train Loss: 0.1218, Val Loss: 0.1281, Val Acc: 0.9623
Epoch 100/100, Train Loss: 0.1145, Val Loss: 0.1226, Val Acc: 0.9623
Fold 3 - Validation Accuracy: 0.9623

Training fold 4/10:

Epoch 10/100, Train Loss: 0.2954, Val Loss: 0.3523, Val Acc: 0.8396
Epoch 20/100, Train Loss: 0.2320, Val Loss: 0.2790, Val Acc: 0.8868
Epoch 30/100, Train Loss: 0.1995, Val Loss: 0.2337, Val Acc: 0.8962
Epoch 40/100, Train Loss: 0.1773, Val Loss: 0.2123, Val Acc: 0.8962
Epoch 50/100, Train Loss: 0.1597, Val Loss: 0.1868, Val Acc: 0.9245
Epoch 60/100, Train Loss: 0.1466, Val Loss: 0.1719, Val Acc: 0.9151

Epoch 70/100, Train Loss: 0.1375, Val Loss: 0.1661, Val Acc: 0.9245
Epoch 80/100, Train Loss: 0.1289, Val Loss: 0.1460, Val Acc: 0.9340
Epoch 90/100, Train Loss: 0.1204, Val Loss: 0.1442, Val Acc: 0.9528
Epoch 100/100, Train Loss: 0.1150, Val Loss: 0.1439, Val Acc: 0.9528
Fold 4 - Validation Accuracy: 0.9528

Training fold 5/10:

Epoch 10/100, Train Loss: 0.3264, Val Loss: 0.2633, Val Acc: 0.8962
Epoch 20/100, Train Loss: 0.2635, Val Loss: 0.2083, Val Acc: 0.9057
Epoch 30/100, Train Loss: 0.2141, Val Loss: 0.1766, Val Acc: 0.9151
Epoch 40/100, Train Loss: 0.1861, Val Loss: 0.1576, Val Acc: 0.9434
Epoch 50/100, Train Loss: 0.1682, Val Loss: 0.1437, Val Acc: 0.9434
Epoch 60/100, Train Loss: 0.1539, Val Loss: 0.1344, Val Acc: 0.9528
Epoch 70/100, Train Loss: 0.1406, Val Loss: 0.1302, Val Acc: 0.9623
Epoch 80/100, Train Loss: 0.1340, Val Loss: 0.1247, Val Acc: 0.9623
Epoch 90/100, Train Loss: 0.1245, Val Loss: 0.1215, Val Acc: 0.9623
Epoch 100/100, Train Loss: 0.1186, Val Loss: 0.1197, Val Acc: 0.9623
Fold 5 - Validation Accuracy: 0.9623

Training fold 6/10:

Epoch 10/100, Train Loss: 0.3111, Val Loss: 0.3690, Val Acc: 0.8190
Epoch 20/100, Train Loss: 0.2427, Val Loss: 0.2997, Val Acc: 0.8762
Epoch 30/100, Train Loss: 0.2025, Val Loss: 0.2500, Val Acc: 0.8952
Epoch 40/100, Train Loss: 0.1741, Val Loss: 0.2246, Val Acc: 0.8952
Epoch 50/100, Train Loss: 0.1553, Val Loss: 0.2095, Val Acc: 0.8952
Epoch 60/100, Train Loss: 0.1402, Val Loss: 0.1976, Val Acc: 0.9048
Epoch 70/100, Train Loss: 0.1310, Val Loss: 0.1856, Val Acc: 0.9048
Epoch 80/100, Train Loss: 0.1208, Val Loss: 0.1788, Val Acc: 0.9238
Epoch 90/100, Train Loss: 0.1148, Val Loss: 0.1783, Val Acc: 0.9238
Epoch 100/100, Train Loss: 0.1087, Val Loss: 0.1729, Val Acc: 0.9143
Fold 6 - Validation Accuracy: 0.9143

Training fold 7/10:

Epoch 10/100, Train Loss: 0.3057, Val Loss: 0.2293, Val Acc: 0.9429
Epoch 20/100, Train Loss: 0.2413, Val Loss: 0.1612, Val Acc: 0.9524
Epoch 30/100, Train Loss: 0.2011, Val Loss: 0.1275, Val Acc: 0.9714
Epoch 40/100, Train Loss: 0.1767, Val Loss: 0.1072, Val Acc: 0.9714
Epoch 50/100, Train Loss: 0.1540, Val Loss: 0.0966, Val Acc: 0.9714
Epoch 60/100, Train Loss: 0.1421, Val Loss: 0.0895, Val Acc: 0.9714
Epoch 70/100, Train Loss: 0.1335, Val Loss: 0.0831, Val Acc: 0.9714
Epoch 80/100, Train Loss: 0.1233, Val Loss: 0.0840, Val Acc: 0.9619

Epoch 90/100, Train Loss: 0.1153, Val Loss: 0.0772, Val Acc: 0.9714
Early stopping triggered at epoch 93
Fold 7 - Validation Accuracy: 0.9619

Training fold 8/10:

Epoch 10/100, Train Loss: 0.3196, Val Loss: 0.3093, Val Acc: 0.8857
Epoch 20/100, Train Loss: 0.2482, Val Loss: 0.2311, Val Acc: 0.9238
Epoch 30/100, Train Loss: 0.2095, Val Loss: 0.1864, Val Acc: 0.9524
Epoch 40/100, Train Loss: 0.1845, Val Loss: 0.1640, Val Acc: 0.9524
Epoch 50/100, Train Loss: 0.1671, Val Loss: 0.1442, Val Acc: 0.9619
Epoch 60/100, Train Loss: 0.1551, Val Loss: 0.1313, Val Acc: 0.9714
Epoch 70/100, Train Loss: 0.1429, Val Loss: 0.1227, Val Acc: 0.9810
Epoch 80/100, Train Loss: 0.1353, Val Loss: 0.1133, Val Acc: 0.9810
Epoch 90/100, Train Loss: 0.1266, Val Loss: 0.1081, Val Acc: 0.9810
Epoch 100/100, Train Loss: 0.1197, Val Loss: 0.1045, Val Acc: 0.9714
Fold 8 - Validation Accuracy: 0.9714

Training fold 9/10:

Epoch 10/100, Train Loss: 0.3547, Val Loss: 0.3093, Val Acc: 0.8286
Epoch 20/100, Train Loss: 0.2667, Val Loss: 0.2373, Val Acc: 0.8667
Epoch 30/100, Train Loss: 0.2215, Val Loss: 0.2012, Val Acc: 0.8952
Epoch 40/100, Train Loss: 0.1954, Val Loss: 0.1748, Val Acc: 0.9143
Epoch 50/100, Train Loss: 0.1824, Val Loss: 0.1562, Val Acc: 0.9238
Epoch 60/100, Train Loss: 0.1616, Val Loss: 0.1428, Val Acc: 0.9238
Epoch 70/100, Train Loss: 0.1505, Val Loss: 0.1305, Val Acc: 0.9333
Epoch 80/100, Train Loss: 0.1412, Val Loss: 0.1217, Val Acc: 0.9429
Epoch 90/100, Train Loss: 0.1350, Val Loss: 0.1149, Val Acc: 0.9524
Epoch 100/100, Train Loss: 0.1284, Val Loss: 0.1092, Val Acc: 0.9524
Fold 9 - Validation Accuracy: 0.9524

Training fold 10/10:

Epoch 10/100, Train Loss: 0.3270, Val Loss: 0.3237, Val Acc: 0.8762
Epoch 20/100, Train Loss: 0.2525, Val Loss: 0.2552, Val Acc: 0.9048
Epoch 30/100, Train Loss: 0.2137, Val Loss: 0.2117, Val Acc: 0.9238
Epoch 40/100, Train Loss: 0.1872, Val Loss: 0.1809, Val Acc: 0.9238
Epoch 50/100, Train Loss: 0.1681, Val Loss: 0.1597, Val Acc: 0.9429
Epoch 60/100, Train Loss: 0.1539, Val Loss: 0.1471, Val Acc: 0.9524
Epoch 70/100, Train Loss: 0.1428, Val Loss: 0.1355, Val Acc: 0.9619
Epoch 80/100, Train Loss: 0.1341, Val Loss: 0.1270, Val Acc: 0.9619
Epoch 90/100, Train Loss: 0.1265, Val Loss: 0.1212, Val Acc: 0.9619
Epoch 100/100, Train Loss: 0.1199, Val Loss: 0.1159, Val Acc: 0.9619

Fold 10 - Validation Accuracy: 0.9619

MLP (Systolic BP) 10-Fold CV Results:

Mean Accuracy: 0.9498

Evaluating Pulse Pressure Model (X2 features)

Training fold 1/10:

Epoch 10/100, Train Loss: 0.3180, Val Loss: 0.3660, Val Acc: 0.7736

Epoch 20/100, Train Loss: 0.2519, Val Loss: 0.3173, Val Acc: 0.8491

Epoch 30/100, Train Loss: 0.2164, Val Loss: 0.2763, Val Acc: 0.9057

Epoch 40/100, Train Loss: 0.1946, Val Loss: 0.2434, Val Acc: 0.9151

Epoch 50/100, Train Loss: 0.1791, Val Loss: 0.2224, Val Acc: 0.9245

Epoch 60/100, Train Loss: 0.1633, Val Loss: 0.2064, Val Acc: 0.9340

Epoch 70/100, Train Loss: 0.1528, Val Loss: 0.1951, Val Acc: 0.9340

Epoch 80/100, Train Loss: 0.1436, Val Loss: 0.1789, Val Acc: 0.9340

Epoch 90/100, Train Loss: 0.1349, Val Loss: 0.1712, Val Acc: 0.9528

Epoch 100/100, Train Loss: 0.1264, Val Loss: 0.1576, Val Acc: 0.9528

Fold 1 - Validation Accuracy: 0.9528

Training fold 2/10:

Epoch 10/100, Train Loss: 0.2772, Val Loss: 0.3936, Val Acc: 0.8491

Early stopping triggered at epoch 18

Fold 2 - Validation Accuracy: 0.8962

Training fold 3/10:

Epoch 10/100, Train Loss: 0.2984, Val Loss: 0.3024, Val Acc: 0.8679

Epoch 20/100, Train Loss: 0.2355, Val Loss: 0.2338, Val Acc: 0.8868

Epoch 30/100, Train Loss: 0.1991, Val Loss: 0.1957, Val Acc: 0.9434

Epoch 40/100, Train Loss: 0.1749, Val Loss: 0.1691, Val Acc: 0.9434

Epoch 50/100, Train Loss: 0.1574, Val Loss: 0.1523, Val Acc: 0.9528

Epoch 60/100, Train Loss: 0.1447, Val Loss: 0.1433, Val Acc: 0.9623

Epoch 70/100, Train Loss: 0.1344, Val Loss: 0.1337, Val Acc: 0.9623

Epoch 80/100, Train Loss: 0.1246, Val Loss: 0.1310, Val Acc: 0.9623

Epoch 90/100, Train Loss: 0.1208, Val Loss: 0.1246, Val Acc: 0.9623

Epoch 100/100, Train Loss: 0.1111, Val Loss: 0.1171, Val Acc: 0.9623

Fold 3 - Validation Accuracy: 0.9623

Training fold 4/10:

Epoch 10/100, Train Loss: 0.3097, Val Loss: 0.3637, Val Acc: 0.8302

Epoch 20/100, Train Loss: 0.2401, Val Loss: 0.2826, Val Acc: 0.8679

Epoch 30/100, Train Loss: 0.1977, Val Loss: 0.2367, Val Acc: 0.8962

Epoch 40/100, Train Loss: 0.1691, Val Loss: 0.2020, Val Acc: 0.9057
Epoch 50/100, Train Loss: 0.1519, Val Loss: 0.1784, Val Acc: 0.9151
Epoch 60/100, Train Loss: 0.1382, Val Loss: 0.1667, Val Acc: 0.9245
Epoch 70/100, Train Loss: 0.1302, Val Loss: 0.1508, Val Acc: 0.9434
Epoch 80/100, Train Loss: 0.1222, Val Loss: 0.1481, Val Acc: 0.9434
Epoch 90/100, Train Loss: 0.1151, Val Loss: 0.1380, Val Acc: 0.9434
Epoch 100/100, Train Loss: 0.1095, Val Loss: 0.1295, Val Acc: 0.9528
Fold 4 - Validation Accuracy: 0.9528

Training fold 5/10:

Epoch 10/100, Train Loss: 0.3338, Val Loss: 0.2725, Val Acc: 0.9057
Epoch 20/100, Train Loss: 0.2612, Val Loss: 0.2105, Val Acc: 0.9245
Epoch 30/100, Train Loss: 0.2122, Val Loss: 0.1787, Val Acc: 0.9340
Epoch 40/100, Train Loss: 0.1822, Val Loss: 0.1595, Val Acc: 0.9340
Epoch 50/100, Train Loss: 0.1671, Val Loss: 0.1469, Val Acc: 0.9434
Epoch 60/100, Train Loss: 0.1498, Val Loss: 0.1354, Val Acc: 0.9528
Epoch 70/100, Train Loss: 0.1369, Val Loss: 0.1318, Val Acc: 0.9528
Epoch 80/100, Train Loss: 0.1302, Val Loss: 0.1256, Val Acc: 0.9528
Epoch 90/100, Train Loss: 0.1242, Val Loss: 0.1226, Val Acc: 0.9528
Epoch 100/100, Train Loss: 0.1164, Val Loss: 0.1199, Val Acc: 0.9528
Fold 5 - Validation Accuracy: 0.9528

Training fold 6/10:

Epoch 10/100, Train Loss: 0.3133, Val Loss: 0.3608, Val Acc: 0.8190
Epoch 20/100, Train Loss: 0.2421, Val Loss: 0.2946, Val Acc: 0.8762
Epoch 30/100, Train Loss: 0.1991, Val Loss: 0.2501, Val Acc: 0.8952
Epoch 40/100, Train Loss: 0.1724, Val Loss: 0.2236, Val Acc: 0.8952
Epoch 50/100, Train Loss: 0.1513, Val Loss: 0.2092, Val Acc: 0.9048
Epoch 60/100, Train Loss: 0.1392, Val Loss: 0.1987, Val Acc: 0.9048
Epoch 70/100, Train Loss: 0.1300, Val Loss: 0.1913, Val Acc: 0.9048
Epoch 80/100, Train Loss: 0.1225, Val Loss: 0.1868, Val Acc: 0.9048
Epoch 90/100, Train Loss: 0.1162, Val Loss: 0.1841, Val Acc: 0.9048
Epoch 100/100, Train Loss: 0.1109, Val Loss: 0.1772, Val Acc: 0.9048
Fold 6 - Validation Accuracy: 0.9048

Training fold 7/10:

Epoch 10/100, Train Loss: 0.3216, Val Loss: 0.2637, Val Acc: 0.9143
Epoch 20/100, Train Loss: 0.2515, Val Loss: 0.1826, Val Acc: 0.9429
Epoch 30/100, Train Loss: 0.2074, Val Loss: 0.1412, Val Acc: 0.9619
Epoch 40/100, Train Loss: 0.1782, Val Loss: 0.1188, Val Acc: 0.9714
Epoch 50/100, Train Loss: 0.1566, Val Loss: 0.1049, Val Acc: 0.9714

Epoch 60/100, Train Loss: 0.1415, Val Loss: 0.0957, Val Acc: 0.9714
Epoch 70/100, Train Loss: 0.1326, Val Loss: 0.0910, Val Acc: 0.9714
Epoch 80/100, Train Loss: 0.1265, Val Loss: 0.0897, Val Acc: 0.9619
Epoch 90/100, Train Loss: 0.1197, Val Loss: 0.0836, Val Acc: 0.9619
Epoch 100/100, Train Loss: 0.1129, Val Loss: 0.0811, Val Acc: 0.9619
Fold 7 - Validation Accuracy: 0.9619

Training fold 8/10:

Epoch 10/100, Train Loss: 0.3007, Val Loss: 0.2847, Val Acc: 0.8857
Epoch 20/100, Train Loss: 0.2356, Val Loss: 0.2176, Val Acc: 0.9143
Epoch 30/100, Train Loss: 0.1995, Val Loss: 0.1829, Val Acc: 0.9619
Epoch 40/100, Train Loss: 0.1766, Val Loss: 0.1544, Val Acc: 0.9619
Epoch 50/100, Train Loss: 0.1586, Val Loss: 0.1362, Val Acc: 0.9619
Epoch 60/100, Train Loss: 0.1453, Val Loss: 0.1259, Val Acc: 0.9714
Epoch 70/100, Train Loss: 0.1362, Val Loss: 0.1163, Val Acc: 0.9810
Epoch 80/100, Train Loss: 0.1257, Val Loss: 0.1104, Val Acc: 0.9810
Epoch 90/100, Train Loss: 0.1212, Val Loss: 0.1036, Val Acc: 0.9810
Epoch 100/100, Train Loss: 0.1167, Val Loss: 0.0998, Val Acc: 0.9810
Fold 8 - Validation Accuracy: 0.9810

Training fold 9/10:

Epoch 10/100, Train Loss: 0.3277, Val Loss: 0.2773, Val Acc: 0.8476
Epoch 20/100, Train Loss: 0.2451, Val Loss: 0.2165, Val Acc: 0.9048
Epoch 30/100, Train Loss: 0.2070, Val Loss: 0.1815, Val Acc: 0.9143
Epoch 40/100, Train Loss: 0.1809, Val Loss: 0.1586, Val Acc: 0.9238
Epoch 50/100, Train Loss: 0.1640, Val Loss: 0.1397, Val Acc: 0.9238
Epoch 60/100, Train Loss: 0.1498, Val Loss: 0.1290, Val Acc: 0.9238
Epoch 70/100, Train Loss: 0.1459, Val Loss: 0.1190, Val Acc: 0.9429
Epoch 80/100, Train Loss: 0.1338, Val Loss: 0.1130, Val Acc: 0.9524
Epoch 90/100, Train Loss: 0.1269, Val Loss: 0.1069, Val Acc: 0.9524
Epoch 100/100, Train Loss: 0.1203, Val Loss: 0.1019, Val Acc: 0.9619
Fold 9 - Validation Accuracy: 0.9619

Training fold 10/10:

Epoch 10/100, Train Loss: 0.3253, Val Loss: 0.3127, Val Acc: 0.8667
Epoch 20/100, Train Loss: 0.2478, Val Loss: 0.2454, Val Acc: 0.9238
Epoch 30/100, Train Loss: 0.2113, Val Loss: 0.2029, Val Acc: 0.9333
Epoch 40/100, Train Loss: 0.1872, Val Loss: 0.1773, Val Acc: 0.9429
Epoch 50/100, Train Loss: 0.1694, Val Loss: 0.1571, Val Acc: 0.9524
Epoch 60/100, Train Loss: 0.1563, Val Loss: 0.1425, Val Acc: 0.9524
Epoch 70/100, Train Loss: 0.1448, Val Loss: 0.1365, Val Acc: 0.9429

Epoch 80/100, Train Loss: 0.1348, Val Loss: 0.1296, Val Acc: 0.9429
 Epoch 90/100, Train Loss: 0.1275, Val Loss: 0.1282, Val Acc: 0.9429
 Epoch 100/100, Train Loss: 0.1204, Val Loss: 0.1276, Val Acc: 0.9524
 Fold 10 - Validation Accuracy: 0.9524

MLP (Pulse Pressure) 10-Fold CV Results:
 Mean Accuracy: 0.9479

Very high accuracy and really good overall performance is observed from the Pytorch version of the baseline model however Overfitting is suspected since such a performance was not expected given that the initial models of SVM and MLP (SKLearn) did not show such high performance.

Test set Evaluation of Pytorch version

```
def evaluate_test_set(model, X_test, y_test): model.eval() with torch.no_grad(): X_test_tensor = torch.FloatTensor(X_test) y_test_tensor = torch.FloatTensor(y_test).view(-1, 1)
```

```
    test_outputs = model(X_test_tensor)
    test_preds = (torch.sigmoid(test_outputs) > 0.5).float()
    test_accuracy = (test_preds == y_test_tensor).sum().item() / len(y_test)

    return test_accuracy
```

Add test set evaluation

```
print("Test Set Evaluation") X1_test_acc = evaluate_test_set(basemodel_X1, X1_test_scaled, y1_test.values) X2_test_acc = evaluate_test_set(basemodel_X2, X2_test_scaled, y2_test.values)
```

```
print(f"MLP (Systolic BP) Test Accuracy: {X1_test_acc:.4f}") print(f"MLP (Pulse Pressure) Test Accuracy: {X2_test_acc:.4f}")
```

Test Set Evaluation MLP (Systolic BP) Test Accuracy: 0.9318 MLP (Pulse Pressure) Test Accuracy: 0.9394

PyTorch Implementation (Reduced features model)

```
In [57]: # Defining input dimensions for reduced features
input_dim_reduced = X_reduced_train_scaled.shape[1]

# Create the model using the same BaseMLP architecture
basemodel_reduced = BaseMLP(input_dim_reduced)

# Optimizer with the same hyperparameters as base model
optimizer_reduced = optim.Adam(basemodel_reduced.parameters(), lr=0.001, weight_decay=0.001)

# Create tensor datasets and data loaders for the reduced model
train_dataset_reduced = TensorDataset(torch.FloatTensor(X_reduced_train_scaled), torch.FloatTensor(y_reduced_train.values).view(-1))
train_loader_reduced = DataLoader(train_dataset_reduced, batch_size=batch_size, shuffle=True)

In [58]: # Run k-fold cross-validation on the reduced features model
print("Evaluating MLP with Reduced Features")
reduced_cv_acc, reduced_fold_accs = kfold_cv(
    dataset=train_dataset_reduced,
    basemodel=basemodel_reduced,
    criterion=criterion,
    optimizer=optimizer_reduced,
    feature_name="MLP (Reduced Features)",
    n_splits=10,
    max_iter=100,
    batch_size=32,
    patience=10
)

print(f"MLP (Reduced Features) CV Accuracy: {reduced_cv_acc:.4f}")
```

Evaluating MLP with Reduced Features

Training fold 1/10:

Epoch 10/100, Train Loss: 0.2947, Val Loss: 0.3525, Val Acc: 0.8302
Epoch 20/100, Train Loss: 0.2264, Val Loss: 0.2639, Val Acc: 0.9057
Epoch 30/100, Train Loss: 0.1899, Val Loss: 0.2097, Val Acc: 0.9340
Epoch 40/100, Train Loss: 0.1688, Val Loss: 0.1745, Val Acc: 0.9434
Epoch 50/100, Train Loss: 0.1526, Val Loss: 0.1531, Val Acc: 0.9434
Epoch 60/100, Train Loss: 0.1409, Val Loss: 0.1382, Val Acc: 0.9623
Epoch 70/100, Train Loss: 0.1308, Val Loss: 0.1275, Val Acc: 0.9623
Epoch 80/100, Train Loss: 0.1233, Val Loss: 0.1204, Val Acc: 0.9623
Epoch 90/100, Train Loss: 0.1165, Val Loss: 0.1140, Val Acc: 0.9528
Epoch 100/100, Train Loss: 0.1104, Val Loss: 0.1126, Val Acc: 0.9623
Fold 1 - Validation Accuracy: 0.9623

Training fold 2/10:

Epoch 10/100, Train Loss: 0.2670, Val Loss: 0.3914, Val Acc: 0.8585
Early stopping triggered at epoch 18
Fold 2 - Validation Accuracy: 0.8962

Training fold 3/10:

Epoch 10/100, Train Loss: 0.3177, Val Loss: 0.3284, Val Acc: 0.8585
Epoch 20/100, Train Loss: 0.2376, Val Loss: 0.2359, Val Acc: 0.9151
Epoch 30/100, Train Loss: 0.1932, Val Loss: 0.1912, Val Acc: 0.9340
Epoch 40/100, Train Loss: 0.1739, Val Loss: 0.1631, Val Acc: 0.9340
Epoch 50/100, Train Loss: 0.1503, Val Loss: 0.1497, Val Acc: 0.9623
Epoch 60/100, Train Loss: 0.1389, Val Loss: 0.1369, Val Acc: 0.9623
Epoch 70/100, Train Loss: 0.1300, Val Loss: 0.1302, Val Acc: 0.9623
Epoch 80/100, Train Loss: 0.1231, Val Loss: 0.1270, Val Acc: 0.9623
Epoch 90/100, Train Loss: 0.1175, Val Loss: 0.1253, Val Acc: 0.9623
Epoch 100/100, Train Loss: 0.1128, Val Loss: 0.1229, Val Acc: 0.9623
Fold 3 - Validation Accuracy: 0.9623

Training fold 4/10:

Epoch 10/100, Train Loss: 0.2931, Val Loss: 0.3220, Val Acc: 0.8679
Epoch 20/100, Train Loss: 0.2251, Val Loss: 0.2504, Val Acc: 0.8868
Epoch 30/100, Train Loss: 0.1859, Val Loss: 0.2111, Val Acc: 0.9057
Epoch 40/100, Train Loss: 0.1649, Val Loss: 0.1776, Val Acc: 0.9245
Epoch 50/100, Train Loss: 0.1490, Val Loss: 0.1641, Val Acc: 0.9245
Epoch 60/100, Train Loss: 0.1389, Val Loss: 0.1464, Val Acc: 0.9245
Epoch 70/100, Train Loss: 0.1385, Val Loss: 0.1385, Val Acc: 0.9340

Early stopping triggered at epoch 77

Fold 4 - Validation Accuracy: 0.9434

Training fold 5/10:

Epoch 10/100, Train Loss: 0.3300, Val Loss: 0.2592, Val Acc: 0.8962

Epoch 20/100, Train Loss: 0.2525, Val Loss: 0.1917, Val Acc: 0.9151

Epoch 30/100, Train Loss: 0.2030, Val Loss: 0.1619, Val Acc: 0.9528

Epoch 40/100, Train Loss: 0.1743, Val Loss: 0.1455, Val Acc: 0.9434

Epoch 50/100, Train Loss: 0.1545, Val Loss: 0.1345, Val Acc: 0.9528

Epoch 60/100, Train Loss: 0.1414, Val Loss: 0.1286, Val Acc: 0.9623

Epoch 70/100, Train Loss: 0.1322, Val Loss: 0.1241, Val Acc: 0.9623

Epoch 80/100, Train Loss: 0.1253, Val Loss: 0.1216, Val Acc: 0.9623

Epoch 90/100, Train Loss: 0.1208, Val Loss: 0.1188, Val Acc: 0.9623

Epoch 100/100, Train Loss: 0.1151, Val Loss: 0.1163, Val Acc: 0.9623

Fold 5 - Validation Accuracy: 0.9623

Training fold 6/10:

Epoch 10/100, Train Loss: 0.2954, Val Loss: 0.3540, Val Acc: 0.8190

Epoch 20/100, Train Loss: 0.2269, Val Loss: 0.2767, Val Acc: 0.8762

Epoch 30/100, Train Loss: 0.1859, Val Loss: 0.2284, Val Acc: 0.8952

Epoch 40/100, Train Loss: 0.1598, Val Loss: 0.2043, Val Acc: 0.8952

Epoch 50/100, Train Loss: 0.1430, Val Loss: 0.1860, Val Acc: 0.9143

Epoch 60/100, Train Loss: 0.1316, Val Loss: 0.1779, Val Acc: 0.9048

Epoch 70/100, Train Loss: 0.1225, Val Loss: 0.1710, Val Acc: 0.9238

Epoch 80/100, Train Loss: 0.1183, Val Loss: 0.1675, Val Acc: 0.9238

Epoch 90/100, Train Loss: 0.1104, Val Loss: 0.1614, Val Acc: 0.9333

Epoch 100/100, Train Loss: 0.1062, Val Loss: 0.1601, Val Acc: 0.9333

Fold 6 - Validation Accuracy: 0.9333

Training fold 7/10:

Epoch 10/100, Train Loss: 0.3356, Val Loss: 0.2802, Val Acc: 0.9143

Epoch 20/100, Train Loss: 0.2631, Val Loss: 0.2008, Val Acc: 0.9429

Epoch 30/100, Train Loss: 0.2183, Val Loss: 0.1563, Val Acc: 0.9524

Epoch 40/100, Train Loss: 0.1879, Val Loss: 0.1293, Val Acc: 0.9714

Epoch 50/100, Train Loss: 0.1650, Val Loss: 0.1074, Val Acc: 0.9714

Epoch 60/100, Train Loss: 0.1497, Val Loss: 0.0990, Val Acc: 0.9714

Epoch 70/100, Train Loss: 0.1388, Val Loss: 0.0907, Val Acc: 0.9714

Epoch 80/100, Train Loss: 0.1320, Val Loss: 0.0854, Val Acc: 0.9810

Epoch 90/100, Train Loss: 0.1245, Val Loss: 0.0838, Val Acc: 0.9810

Epoch 100/100, Train Loss: 0.1190, Val Loss: 0.0803, Val Acc: 0.9810

Fold 7 - Validation Accuracy: 0.9810

Training fold 8/10:

Epoch 10/100, Train Loss: 0.3128, Val Loss: 0.2988, Val Acc: 0.8762
Epoch 20/100, Train Loss: 0.2412, Val Loss: 0.2245, Val Acc: 0.9238
Epoch 30/100, Train Loss: 0.2000, Val Loss: 0.1811, Val Acc: 0.9619
Epoch 40/100, Train Loss: 0.1753, Val Loss: 0.1556, Val Acc: 0.9619
Epoch 50/100, Train Loss: 0.1583, Val Loss: 0.1394, Val Acc: 0.9619
Epoch 60/100, Train Loss: 0.1468, Val Loss: 0.1282, Val Acc: 0.9619
Epoch 70/100, Train Loss: 0.1384, Val Loss: 0.1202, Val Acc: 0.9619
Epoch 80/100, Train Loss: 0.1333, Val Loss: 0.1145, Val Acc: 0.9619
Epoch 90/100, Train Loss: 0.1275, Val Loss: 0.1110, Val Acc: 0.9619
Epoch 100/100, Train Loss: 0.1237, Val Loss: 0.1071, Val Acc: 0.9714
Fold 8 - Validation Accuracy: 0.9714

Training fold 9/10:

Epoch 10/100, Train Loss: 0.3045, Val Loss: 0.2643, Val Acc: 0.8857
Epoch 20/100, Train Loss: 0.2338, Val Loss: 0.2020, Val Acc: 0.9048
Epoch 30/100, Train Loss: 0.1927, Val Loss: 0.1614, Val Acc: 0.9333
Epoch 40/100, Train Loss: 0.1675, Val Loss: 0.1369, Val Acc: 0.9429
Epoch 50/100, Train Loss: 0.1533, Val Loss: 0.1207, Val Acc: 0.9619
Epoch 60/100, Train Loss: 0.1409, Val Loss: 0.1104, Val Acc: 0.9714
Epoch 70/100, Train Loss: 0.1324, Val Loss: 0.1017, Val Acc: 0.9714
Epoch 80/100, Train Loss: 0.1243, Val Loss: 0.0973, Val Acc: 0.9714
Epoch 90/100, Train Loss: 0.1190, Val Loss: 0.0937, Val Acc: 0.9714
Epoch 100/100, Train Loss: 0.1140, Val Loss: 0.0911, Val Acc: 0.9714
Fold 9 - Validation Accuracy: 0.9714

Training fold 10/10:

Epoch 10/100, Train Loss: 0.3251, Val Loss: 0.3130, Val Acc: 0.8952
Epoch 20/100, Train Loss: 0.2456, Val Loss: 0.2348, Val Acc: 0.9143
Epoch 30/100, Train Loss: 0.1963, Val Loss: 0.1828, Val Acc: 0.9333
Epoch 40/100, Train Loss: 0.1678, Val Loss: 0.1488, Val Acc: 0.9429
Epoch 50/100, Train Loss: 0.1499, Val Loss: 0.1352, Val Acc: 0.9524
Epoch 60/100, Train Loss: 0.1366, Val Loss: 0.1244, Val Acc: 0.9524
Epoch 70/100, Train Loss: 0.1264, Val Loss: 0.1202, Val Acc: 0.9619
Epoch 80/100, Train Loss: 0.1198, Val Loss: 0.1185, Val Acc: 0.9619
Epoch 90/100, Train Loss: 0.1149, Val Loss: 0.1207, Val Acc: 0.9619
Early stopping triggered at epoch 95
Fold 10 - Validation Accuracy: 0.9619

MLP (Reduced Features) 10-Fold CV Results:

MLP (Reduced Features) CV Accuracy: 0.9545

Using Optimized Hyperparameters from SciKit Learn MLP Classifier Grid Search on Pytorch Version with 'Reduced' features

```
In [59]: # From GridSearch (SKLearn MLP Classifier)
# 'activation': 'relu', 'alpha': 0.01, 'early_stopping': True, 'hidden_layer_sizes': (20, 10), 'learning_rate_init': 0.01, 'max_iter': 1000, 'n_jobs': -1, 'random_state': 0, 'verbose': 0, 'warm_start': False
class OptimizedMLP(nn.Module):
    def __init__(self, input_dim, hidden_sizes=(20, 10), output_dim=1):
        super(OptimizedMLP, self).__init__()
        # Hidden Layers
        self.fc1 = nn.Linear(input_dim, hidden_sizes[0]) # First
        self.fc2 = nn.Linear(hidden_sizes[0], hidden_sizes[1]) # Second
        self.fc3 = nn.Linear(hidden_sizes[1], output_dim) # Output
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.fc1(x) # ReLU in first layer
        x = self.relu(x)
        x = self.fc2(x) # ReLU in second layer
        x = self.relu(x)
        x = self.fc3(x) # Output layer (Sigmoid (added during Loss calculation))

        return x
```

```
In [60]: optimized_model_reduced = OptimizedMLP(input_dim_reduced, hidden_sizes=(20, 10))
optimizer_reduced = torch.optim.Adam(optimized_model_reduced.parameters(), lr=0.01, weight_decay=0.01)
```

```
In [69]: # Run cross-validation with optimized model for X2
print("Optimized Model (Reduced Features) with K-Fold CV")
X2_cv_acc, X2_fold_accs = kfold_cv(
    dataset=train_dataset_reduced,
    basemodel=optimized_model_reduced,
    criterion=criterion,
    optimizer=optimizer_X2,
    feature_name="Optimized MLP (Reduced Features)",
    max_iter=500, # max iter from hyperparameters
```

```
patience=10 # Early stopping  
)
```

Optimized Model (Reduced Features) with K-Fold CV

Training fold 1/10:

Epoch 10/500, Train Loss: 0.3682, Val Loss: 0.3938, Val Acc: 0.7736
Epoch 20/500, Train Loss: 0.2421, Val Loss: 0.2844, Val Acc: 0.8774
Epoch 30/500, Train Loss: 0.1813, Val Loss: 0.1956, Val Acc: 0.9434
Epoch 40/500, Train Loss: 0.1529, Val Loss: 0.1535, Val Acc: 0.9434
Epoch 50/500, Train Loss: 0.1343, Val Loss: 0.1299, Val Acc: 0.9434
Epoch 60/500, Train Loss: 0.1237, Val Loss: 0.1149, Val Acc: 0.9717
Epoch 70/500, Train Loss: 0.1132, Val Loss: 0.1064, Val Acc: 0.9717
Epoch 80/500, Train Loss: 0.1069, Val Loss: 0.1007, Val Acc: 0.9717
Epoch 90/500, Train Loss: 0.1019, Val Loss: 0.0961, Val Acc: 0.9717
Epoch 100/500, Train Loss: 0.0956, Val Loss: 0.0937, Val Acc: 0.9623
Epoch 110/500, Train Loss: 0.0918, Val Loss: 0.0931, Val Acc: 0.9623
Epoch 120/500, Train Loss: 0.0892, Val Loss: 0.0893, Val Acc: 0.9528
Epoch 130/500, Train Loss: 0.0858, Val Loss: 0.0855, Val Acc: 0.9717
Epoch 140/500, Train Loss: 0.0826, Val Loss: 0.0861, Val Acc: 0.9717
Early stopping triggered at epoch 140
Fold 1 - Validation Accuracy: 0.9717

Training fold 2/10:

Epoch 10/500, Train Loss: 0.2979, Val Loss: 0.3676, Val Acc: 0.8302
Epoch 20/500, Train Loss: 0.1879, Val Loss: 0.3936, Val Acc: 0.9057
Early stopping triggered at epoch 23
Fold 2 - Validation Accuracy: 0.9151

Training fold 3/10:

Epoch 10/500, Train Loss: 0.3506, Val Loss: 0.3593, Val Acc: 0.8491
Epoch 20/500, Train Loss: 0.2319, Val Loss: 0.2261, Val Acc: 0.9057
Epoch 30/500, Train Loss: 0.1788, Val Loss: 0.1706, Val Acc: 0.9340
Epoch 40/500, Train Loss: 0.1471, Val Loss: 0.1389, Val Acc: 0.9623
Epoch 50/500, Train Loss: 0.1278, Val Loss: 0.1287, Val Acc: 0.9623
Epoch 60/500, Train Loss: 0.1157, Val Loss: 0.1235, Val Acc: 0.9623
Epoch 70/500, Train Loss: 0.1066, Val Loss: 0.1241, Val Acc: 0.9623
Epoch 80/500, Train Loss: 0.0996, Val Loss: 0.1140, Val Acc: 0.9623
Epoch 90/500, Train Loss: 0.0915, Val Loss: 0.1142, Val Acc: 0.9623
Early stopping triggered at epoch 90
Fold 3 - Validation Accuracy: 0.9623

Training fold 4/10:

Epoch 10/500, Train Loss: 0.3091, Val Loss: 0.3347, Val Acc: 0.8774

Epoch 20/500, Train Loss: 0.2056, Val Loss: 0.2315, Val Acc: 0.8868
Epoch 30/500, Train Loss: 0.1561, Val Loss: 0.1803, Val Acc: 0.9151
Epoch 40/500, Train Loss: 0.1308, Val Loss: 0.1411, Val Acc: 0.9340
Epoch 50/500, Train Loss: 0.1185, Val Loss: 0.1212, Val Acc: 0.9434
Epoch 60/500, Train Loss: 0.1072, Val Loss: 0.1097, Val Acc: 0.9528
Epoch 70/500, Train Loss: 0.1025, Val Loss: 0.0995, Val Acc: 0.9717
Epoch 80/500, Train Loss: 0.0977, Val Loss: 0.1028, Val Acc: 0.9623
Epoch 90/500, Train Loss: 0.0936, Val Loss: 0.1000, Val Acc: 0.9717
Epoch 100/500, Train Loss: 0.0881, Val Loss: 0.0940, Val Acc: 0.9717
Early stopping triggered at epoch 108
Fold 4 - Validation Accuracy: 0.9623

Training fold 5/10:

Epoch 10/500, Train Loss: 0.3158, Val Loss: 0.2389, Val Acc: 0.9057
Epoch 20/500, Train Loss: 0.1989, Val Loss: 0.1571, Val Acc: 0.9528
Epoch 30/500, Train Loss: 0.1442, Val Loss: 0.1265, Val Acc: 0.9528
Epoch 40/500, Train Loss: 0.1198, Val Loss: 0.1134, Val Acc: 0.9528
Epoch 50/500, Train Loss: 0.1085, Val Loss: 0.1061, Val Acc: 0.9528
Epoch 60/500, Train Loss: 0.0986, Val Loss: 0.1027, Val Acc: 0.9528
Epoch 70/500, Train Loss: 0.0926, Val Loss: 0.1004, Val Acc: 0.9623
Epoch 80/500, Train Loss: 0.0880, Val Loss: 0.0984, Val Acc: 0.9623
Epoch 90/500, Train Loss: 0.0835, Val Loss: 0.1015, Val Acc: 0.9623
Early stopping triggered at epoch 94
Fold 5 - Validation Accuracy: 0.9623

Training fold 6/10:

Epoch 10/500, Train Loss: 0.3565, Val Loss: 0.4203, Val Acc: 0.7714
Epoch 20/500, Train Loss: 0.2140, Val Loss: 0.2750, Val Acc: 0.8857
Epoch 30/500, Train Loss: 0.1514, Val Loss: 0.1979, Val Acc: 0.8952
Epoch 40/500, Train Loss: 0.1219, Val Loss: 0.1675, Val Acc: 0.9238
Epoch 50/500, Train Loss: 0.1070, Val Loss: 0.1517, Val Acc: 0.9524
Epoch 60/500, Train Loss: 0.0994, Val Loss: 0.1489, Val Acc: 0.9524
Epoch 70/500, Train Loss: 0.0940, Val Loss: 0.1436, Val Acc: 0.9619
Epoch 80/500, Train Loss: 0.0880, Val Loss: 0.1471, Val Acc: 0.9524
Epoch 90/500, Train Loss: 0.0850, Val Loss: 0.1413, Val Acc: 0.9714
Early stopping triggered at epoch 92
Fold 6 - Validation Accuracy: 0.9714

Training fold 7/10:

Epoch 10/500, Train Loss: 0.3352, Val Loss: 0.2770, Val Acc: 0.9333
Epoch 20/500, Train Loss: 0.2195, Val Loss: 0.1525, Val Acc: 0.9524

Epoch 30/500, Train Loss: 0.1553, Val Loss: 0.0925, Val Acc: 0.9714
Epoch 40/500, Train Loss: 0.1284, Val Loss: 0.0751, Val Acc: 0.9714
Epoch 50/500, Train Loss: 0.1140, Val Loss: 0.0675, Val Acc: 0.9905
Epoch 60/500, Train Loss: 0.1056, Val Loss: 0.0623, Val Acc: 0.9810
Early stopping triggered at epoch 69
Fold 7 - Validation Accuracy: 0.9714

Training fold 8/10:

Epoch 10/500, Train Loss: 0.3078, Val Loss: 0.2910, Val Acc: 0.8857
Epoch 20/500, Train Loss: 0.2096, Val Loss: 0.1867, Val Acc: 0.9333
Epoch 30/500, Train Loss: 0.1598, Val Loss: 0.1316, Val Acc: 0.9619
Epoch 40/500, Train Loss: 0.1328, Val Loss: 0.1062, Val Acc: 0.9714
Epoch 50/500, Train Loss: 0.1201, Val Loss: 0.0952, Val Acc: 0.9810
Epoch 60/500, Train Loss: 0.1105, Val Loss: 0.0887, Val Acc: 0.9810
Epoch 70/500, Train Loss: 0.1056, Val Loss: 0.0821, Val Acc: 0.9714
Epoch 80/500, Train Loss: 0.1004, Val Loss: 0.0772, Val Acc: 0.9714
Epoch 90/500, Train Loss: 0.0950, Val Loss: 0.0730, Val Acc: 0.9714
Epoch 100/500, Train Loss: 0.0926, Val Loss: 0.0684, Val Acc: 0.9714
Epoch 110/500, Train Loss: 0.0874, Val Loss: 0.0663, Val Acc: 0.9810
Epoch 120/500, Train Loss: 0.0843, Val Loss: 0.0662, Val Acc: 0.9905
Epoch 130/500, Train Loss: 0.0813, Val Loss: 0.0679, Val Acc: 0.9905
Epoch 140/500, Train Loss: 0.0778, Val Loss: 0.0615, Val Acc: 0.9905
Epoch 150/500, Train Loss: 0.0777, Val Loss: 0.0613, Val Acc: 0.9905
Epoch 160/500, Train Loss: 0.0753, Val Loss: 0.0600, Val Acc: 0.9905
Epoch 170/500, Train Loss: 0.0732, Val Loss: 0.0552, Val Acc: 0.9810
Epoch 180/500, Train Loss: 0.0739, Val Loss: 0.0571, Val Acc: 0.9905
Early stopping triggered at epoch 183
Fold 8 - Validation Accuracy: 0.9905

Training fold 9/10:

Epoch 10/500, Train Loss: 0.2865, Val Loss: 0.2508, Val Acc: 0.8857
Epoch 20/500, Train Loss: 0.1975, Val Loss: 0.1739, Val Acc: 0.9143
Epoch 30/500, Train Loss: 0.1580, Val Loss: 0.1283, Val Acc: 0.9619
Epoch 40/500, Train Loss: 0.1358, Val Loss: 0.1034, Val Acc: 0.9714
Epoch 50/500, Train Loss: 0.1236, Val Loss: 0.0888, Val Acc: 0.9714
Epoch 60/500, Train Loss: 0.1144, Val Loss: 0.0808, Val Acc: 0.9714
Epoch 70/500, Train Loss: 0.1071, Val Loss: 0.0737, Val Acc: 0.9714
Epoch 80/500, Train Loss: 0.1073, Val Loss: 0.0715, Val Acc: 0.9714
Epoch 90/500, Train Loss: 0.0991, Val Loss: 0.0648, Val Acc: 0.9714
Epoch 100/500, Train Loss: 0.0949, Val Loss: 0.0639, Val Acc: 0.9810
Epoch 110/500, Train Loss: 0.0895, Val Loss: 0.0645, Val Acc: 0.9714

Epoch 120/500, Train Loss: 0.0859, Val Loss: 0.0593, Val Acc: 0.9810
 Epoch 130/500, Train Loss: 0.0854, Val Loss: 0.0582, Val Acc: 0.9714
 Epoch 140/500, Train Loss: 0.0821, Val Loss: 0.0566, Val Acc: 0.9810
 Epoch 150/500, Train Loss: 0.0783, Val Loss: 0.0537, Val Acc: 0.9714
 Early stopping triggered at epoch 158
 Fold 9 - Validation Accuracy: 0.9714

Training fold 10/10:

Epoch 10/500, Train Loss: 0.3318, Val Loss: 0.3108, Val Acc: 0.9048
 Epoch 20/500, Train Loss: 0.2426, Val Loss: 0.2241, Val Acc: 0.9143
 Epoch 30/500, Train Loss: 0.1875, Val Loss: 0.1711, Val Acc: 0.9333
 Epoch 40/500, Train Loss: 0.1533, Val Loss: 0.1324, Val Acc: 0.9524
 Epoch 50/500, Train Loss: 0.1329, Val Loss: 0.1114, Val Acc: 0.9714
 Epoch 60/500, Train Loss: 0.1212, Val Loss: 0.1021, Val Acc: 0.9714
 Epoch 70/500, Train Loss: 0.1118, Val Loss: 0.0942, Val Acc: 0.9714
 Epoch 80/500, Train Loss: 0.1041, Val Loss: 0.0942, Val Acc: 0.9810
 Epoch 90/500, Train Loss: 0.0986, Val Loss: 0.0920, Val Acc: 0.9810
 Epoch 100/500, Train Loss: 0.0953, Val Loss: 0.0875, Val Acc: 0.9810
 Epoch 110/500, Train Loss: 0.0901, Val Loss: 0.0878, Val Acc: 0.9810
 Epoch 120/500, Train Loss: 0.0875, Val Loss: 0.0871, Val Acc: 0.9810
 Early stopping triggered at epoch 128
 Fold 10 - Validation Accuracy: 0.9714

Optimized MLP (Reduced Features) 10-Fold CV Results:
 Mean Accuracy: 0.9650

Lower accuracy is observed with the Pytorch version with tuned hyperparameters of 0.9422 against 0.9602. Grid search specific for this model may further optimise this model but we will not attempt it since the Pytorch is generating really good performance already.

Test set evalutaion of Optimised model

```
In [62]: def evaluate_test_set(model, X_test, y_test):
          model.eval()
          with torch.no_grad():
              X_test_tensor = torch.FloatTensor(X_test)
              y_test_tensor = torch.FloatTensor(y_test).view(-1, 1)

              test_outputs = model(X_test_tensor)
              test_preds = (torch.sigmoid(test_outputs) > 0.5).float()
```

```

        test_accuracy = (test_preds == y_test_tensor).sum().item() / len(y_test)

    return test_accuracy

print("Test Set Evaluation of the model with optimized hyperparameters")
reduced_test_acc = evaluate_test_set(optimized_model_reduced, X_reduced_test_scaled, y_reduced_test.values)
print(f"Test Accuracy: {reduced_test_acc :.4f}")

```

Test Set Evaluation of the model with optimized hyperparameters

Test Accuracy: 0.9545

```

In [63]: def get_classification_report(model, X_test, y_test):
    # Set model to evaluation mode
    model.eval()

    # Convert data to PyTorch tensors
    X_test_tensor = torch.FloatTensor(X_test)
    y_test_np = y_test.ravel() # Flatten for sklearn metrics

    with torch.no_grad():
        outputs = model(X_test_tensor)
        probabilities = torch.sigmoid(outputs).numpy().ravel()
        predictions = (probabilities > 0.5).astype(int)

    report = classification_report(y_test_np, predictions)

    return report

# Generate and print the classification report
print("Classification Report for Pytorch MLP with optimised hyperparameters:")
report = get_classification_report(optimized_model_reduced, X_reduced_test_scaled, y_reduced_test.values)
print(report)

```

Classification Report for Pytorch MLP with optimised hyperparameters:

	precision	recall	f1-score	support
0	0.95	0.93	0.94	101
1	0.96	0.97	0.96	163
accuracy			0.95	264
macro avg	0.95	0.95	0.95	264
weighted avg	0.95	0.95	0.95	264

```
In [64]: def plot_confusion_matrix(model, X_test, y_test):
# Set model to evaluation mode
model.eval()

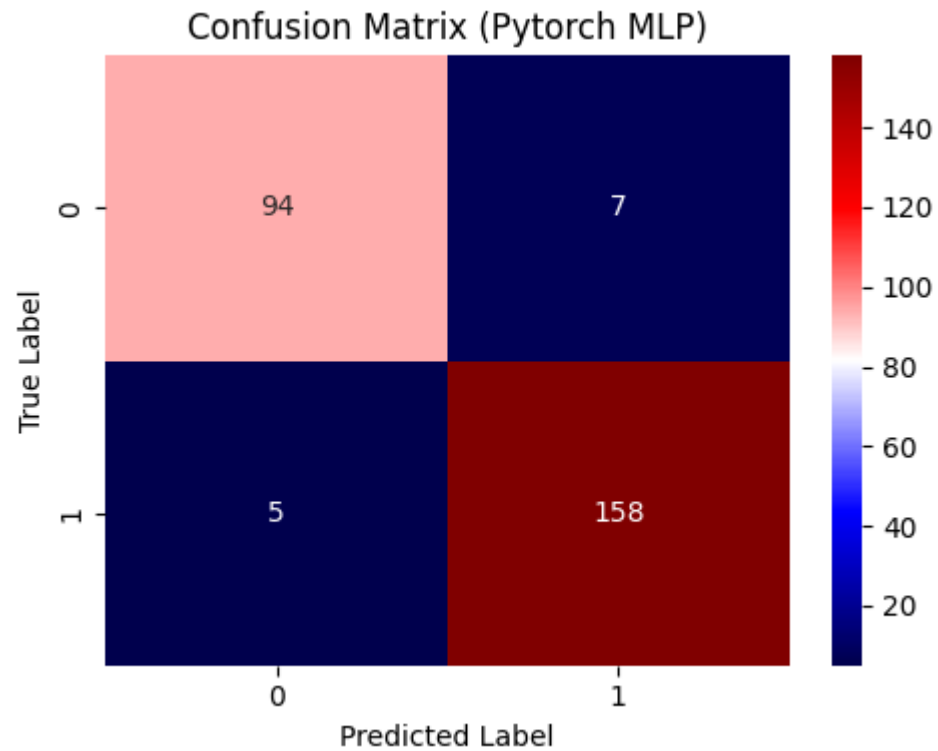
# Convert data to PyTorch tensors
X_test_tensor = torch.FloatTensor(X_test)
y_test_np = y_test.ravel() # Flatten for sklearn metrics

# Get predictions
with torch.no_grad():
    outputs = model(X_test_tensor)
    probabilities = torch.sigmoid(outputs).numpy().ravel()
    predictions = (probabilities > 0.5).astype(int)

# Generate confusion matrix
cm = confusion_matrix(y_test_np, predictions)

# Extract values for reporting
tn, fp, fn, tp = cm.ravel()

# Plot the confusion matrix
plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='seismic', xticklabels=['0', '1'], yticklabels=['0', '1'])
plt.title('Confusion Matrix (Pytorch MLP)')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
return cm
cm = plot_confusion_matrix(optimized_model_reduced, X_reduced_test_scaled, y_reduced_test.values)
```



```
In [65]: def plot_roc_curve(model, X_test, y_test):
# Set model to evaluation mode
model.eval()

# Convert data to PyTorch tensors
X_test_tensor = torch.FloatTensor(X_test)
y_test_np = y_test.ravel() # Changed for sklearn metrics

with torch.no_grad(): # Probabilities calculations
    outputs = model(X_test_tensor)
    probabilities = torch.sigmoid(outputs).numpy().ravel()

# Calculate ROC curve points and AUC
fpr, tpr, thresholds = roc_curve(y_test_np, probabilities)
roc_auc = auc(fpr, tpr)
```

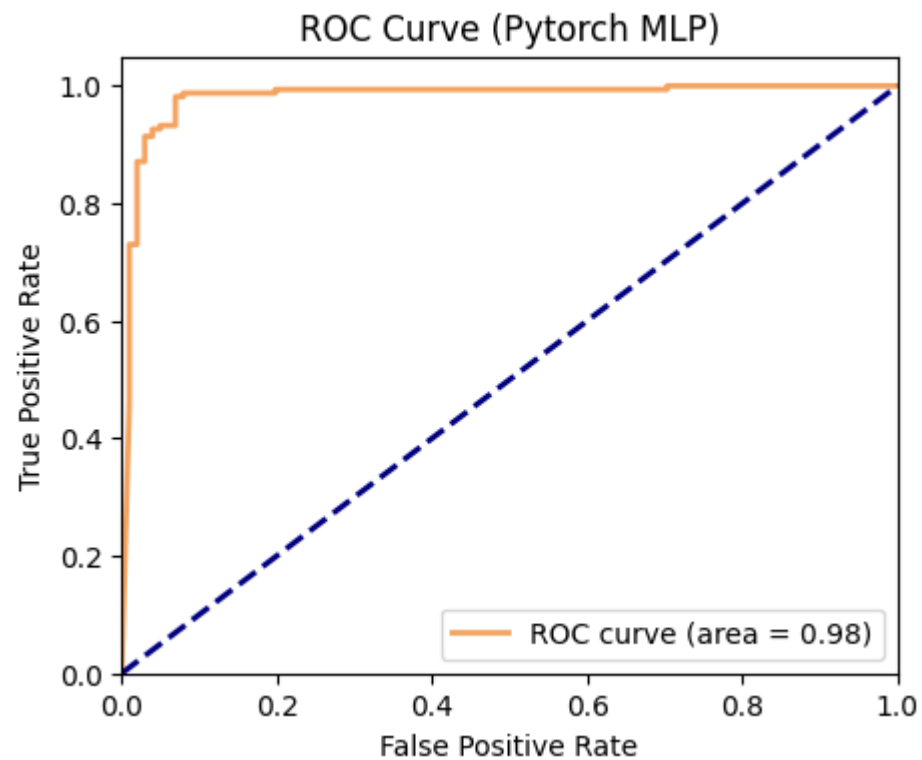
```

# Plot the ROC curve
plt.figure(figsize=(5, 4))
plt.plot(fpr, tpr, color='sandybrown', lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve (Pytorch MLP)')
plt.legend()

return roc_auc

# Generate and plot the ROC curve
roc_auc = plot_roc_curve(optimized_model_reduced, X_reduced_test_scaled, y_reduced_test.values)

```



In []: